



CMPT 413/713: Natural Language Processing

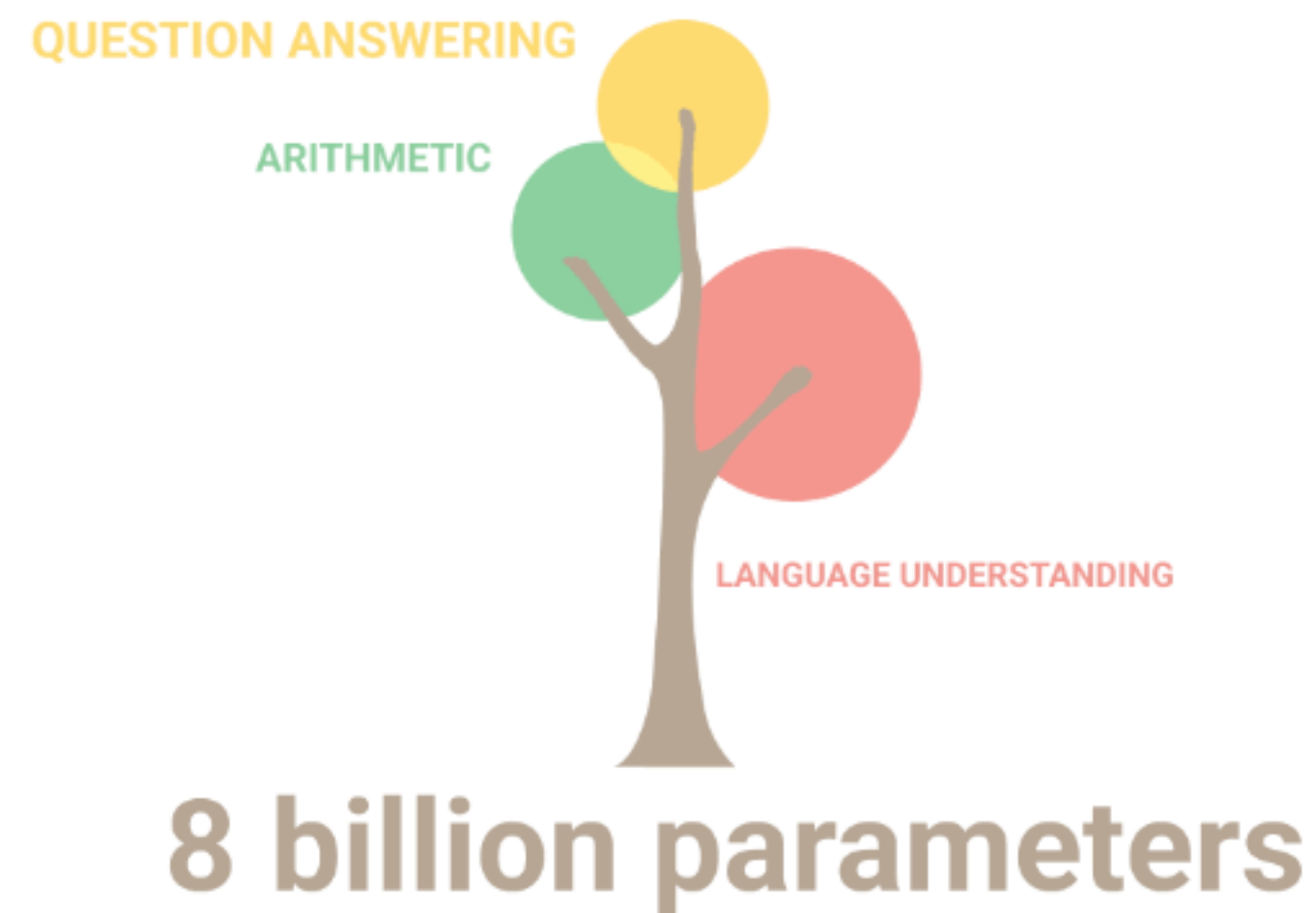
# LLM data and scaling

Spring 2026  
2026-03-09

Some slides adapted from Anoop Sarkar

# Large models

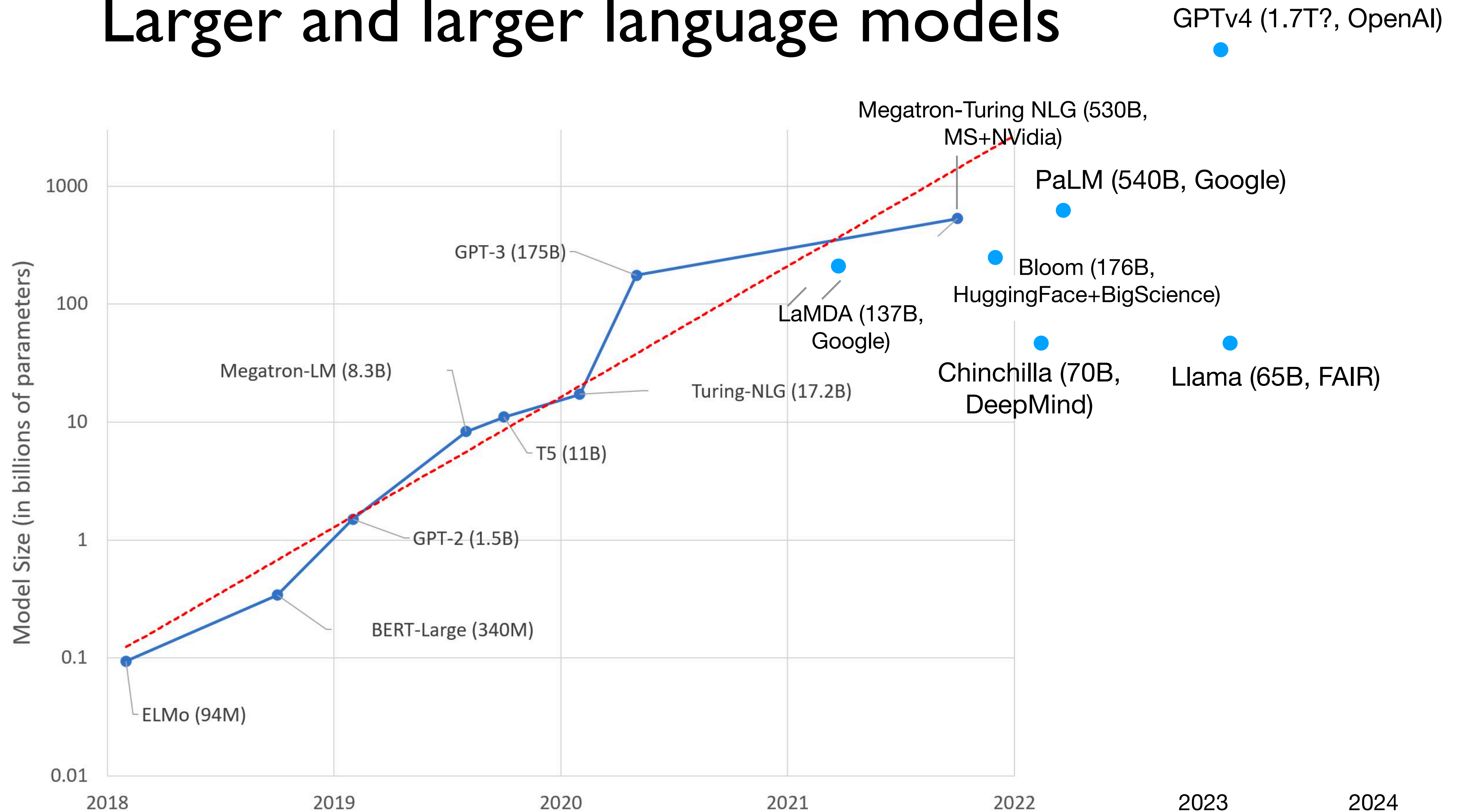
# New capabilities emerge at scale



<https://ai.googleblog.com/2022/04/pathways-language-model-palm-scaling-to.html>



# Larger and larger language models



<https://huggingface.co/blog/large-language-models>

# Data for model training

# Training stages

Piles of  
unlabeled  
text!



Self-supervised training  
LM objective

$$p(w_t | w_{<t})$$

## Pre-training

LM training on large, large  
amount of data

Pre-training can be  
broken into stages (mid-  
training)

Text with  
“instructions”  
and  
“responses”

List three fruit  
Apple, orange,  
banana

Supervised training  
LM objective

$$p(w_t | w_{<t}; \text{prompt})$$

## Instruction-tuning

Supervised fine-tuning for  
instructions

Human  
preference data  
Data about what  
people prefer



Reinforcement  
learning

## Preference optimization

Align to human  
preferences

## Post-training

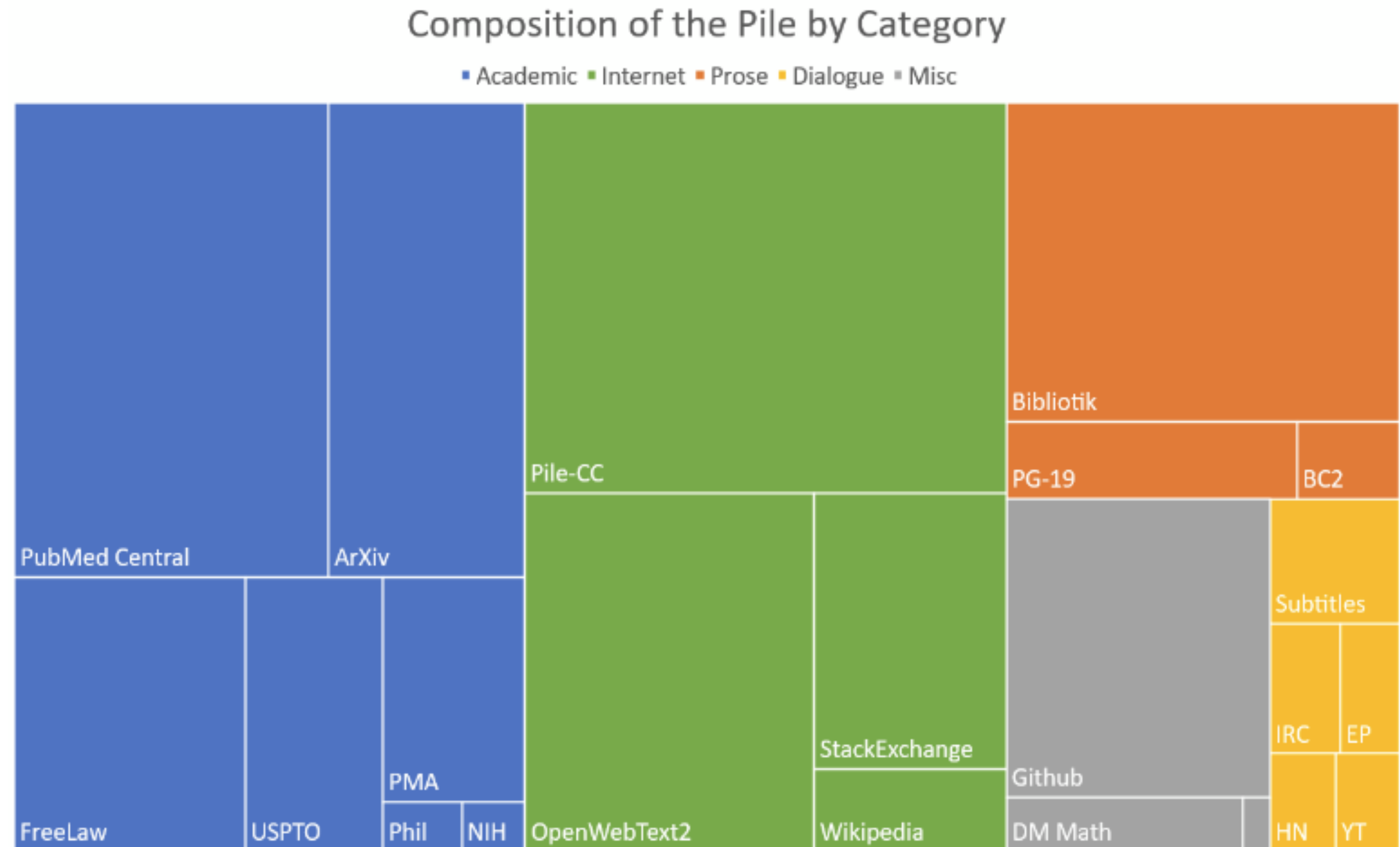
(Can have more iterations)



# The Pile

## Open source data for language modelling

- 800 GB



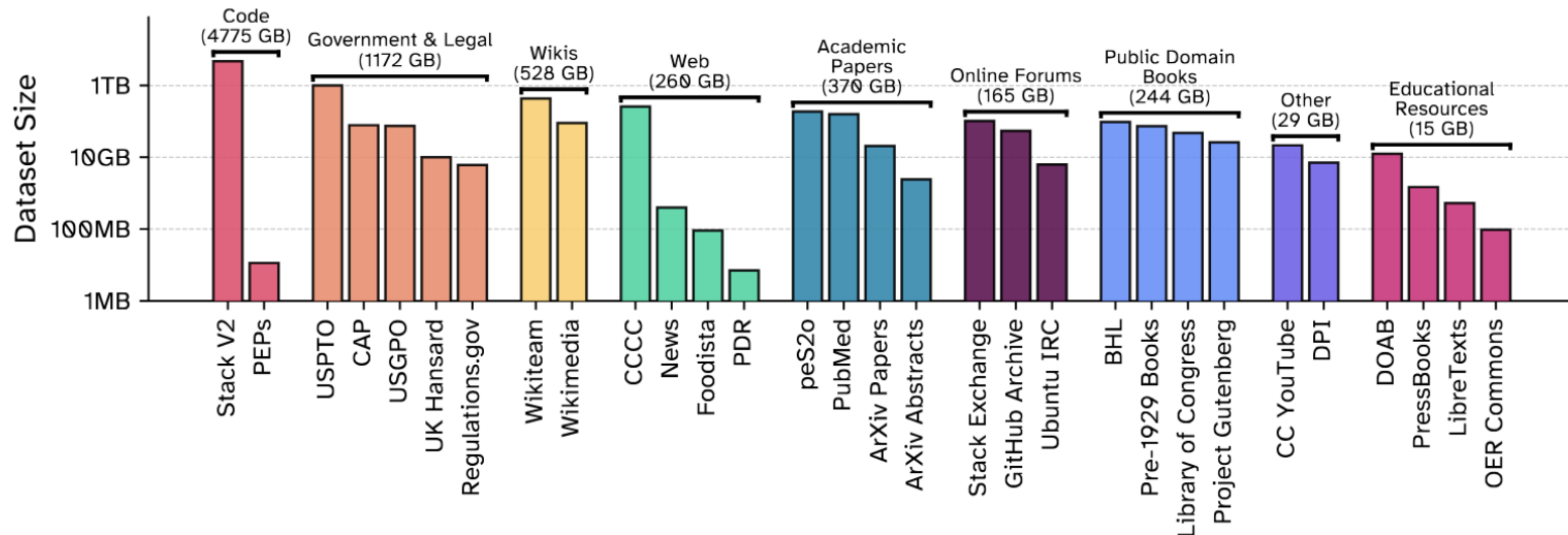
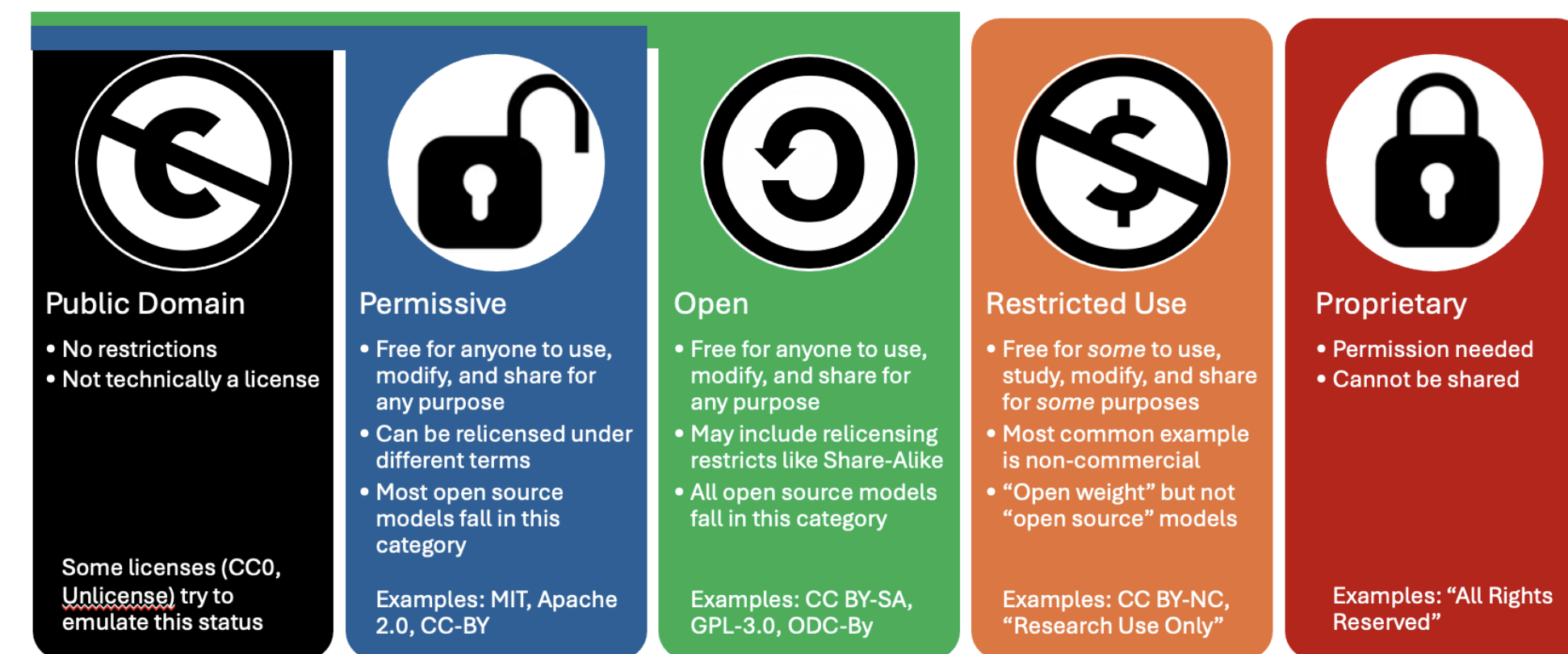
[The Pile: An 800GB Dataset of Diverse Text for Language Modeling \[Gao et al, EleutherAI, 2020\]](#)



# The Common Pile (Comma)

## Open source data for language modelling

- 8TB dataset - public and open license



Collaboration across many institutions (Eleuther AI, HuggingFace, AI2, etc)



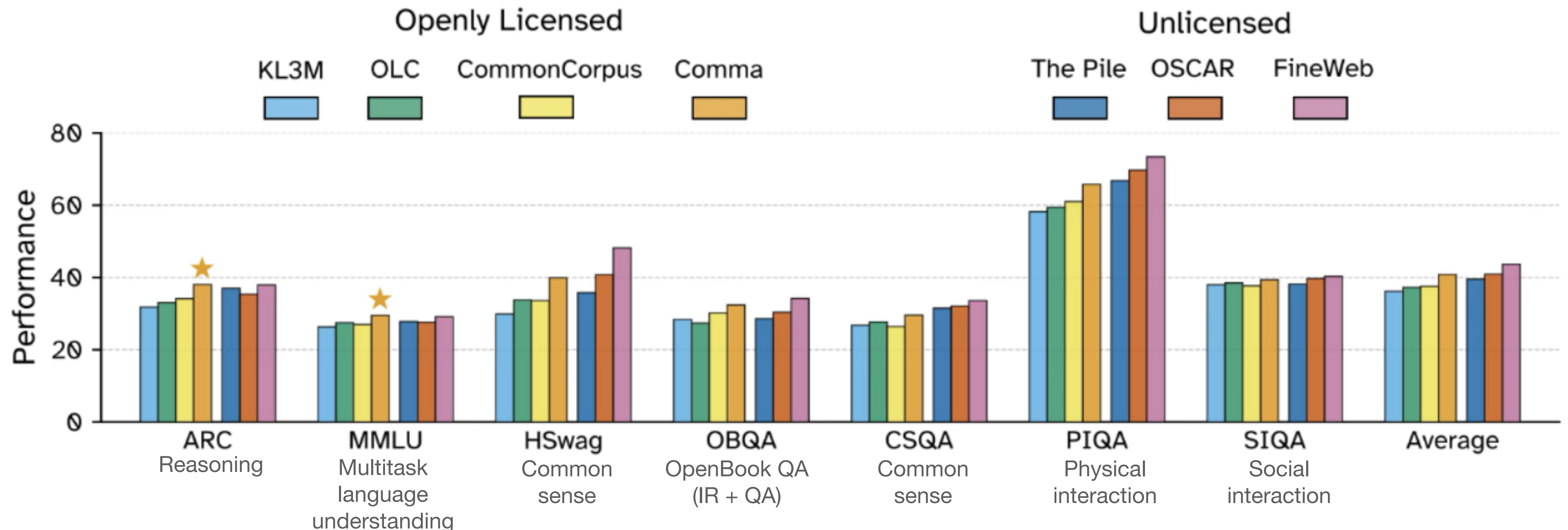
# Comma v0.1

## Open source data for language modelling

- Compare 1.7B model trained on different datasets (28B tokens)

### Benchmarked on QA and reasoning datasets

(star indicates where Comma data outperforms other data)

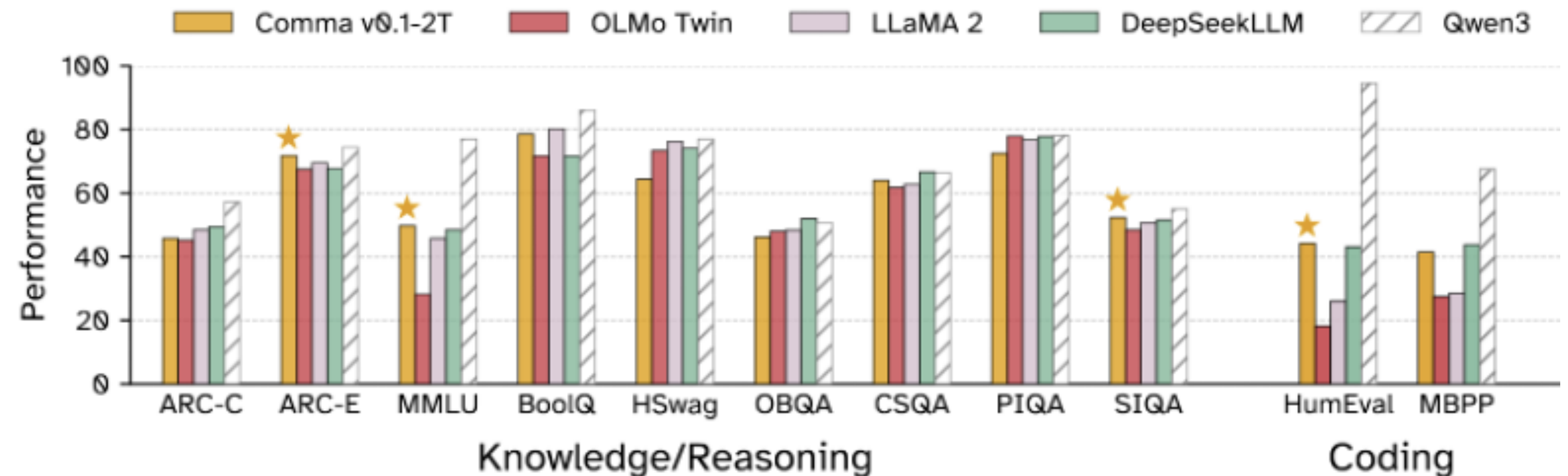
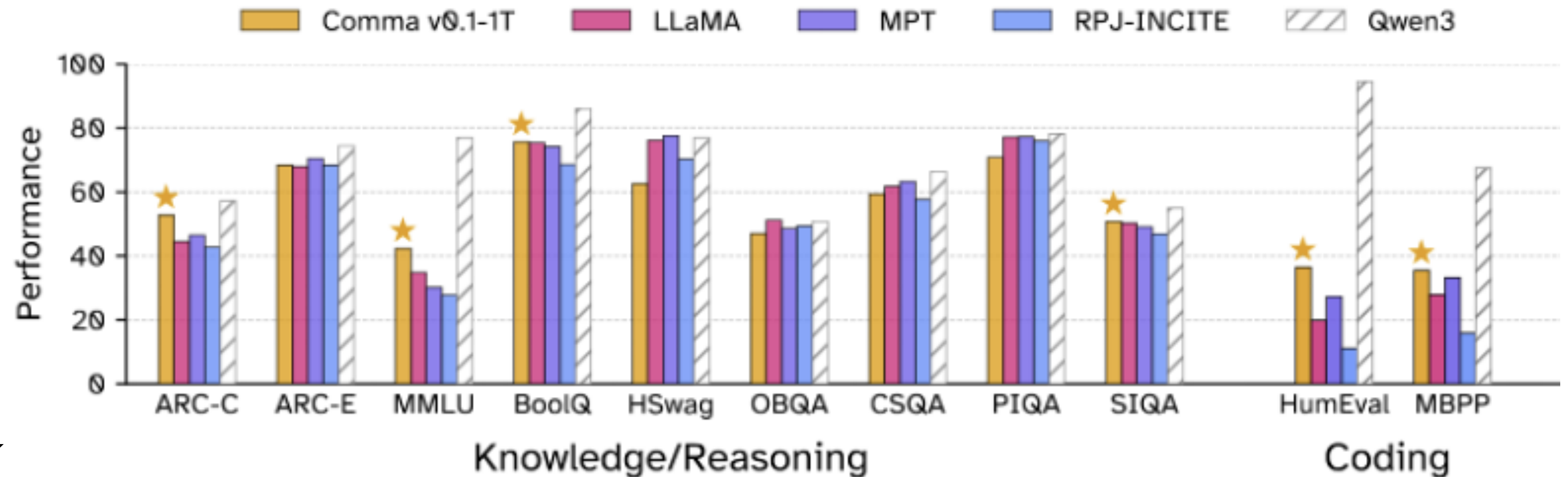


[The Common Pile v0.1: An 8TB Dataset of Public Domain and Openly Licensed Text \[Kandpal et al. 2025\]](#)

# Comma v0.1

- 7B model trained on Comma v0.1
- BPE tokenizer with vocabulary size of 64K (trained on 600GB sample)
- Follow Llama-7B architecture and training

Comparison against similar size models  
(star indicates where Comma model outperforms other compute-matched models)



# Olmo open model

## Open weights, data, checkpoints

- Open-weight models
  - Release final checkpoints
- Open-flow models
  - Release intermediate checkpoints

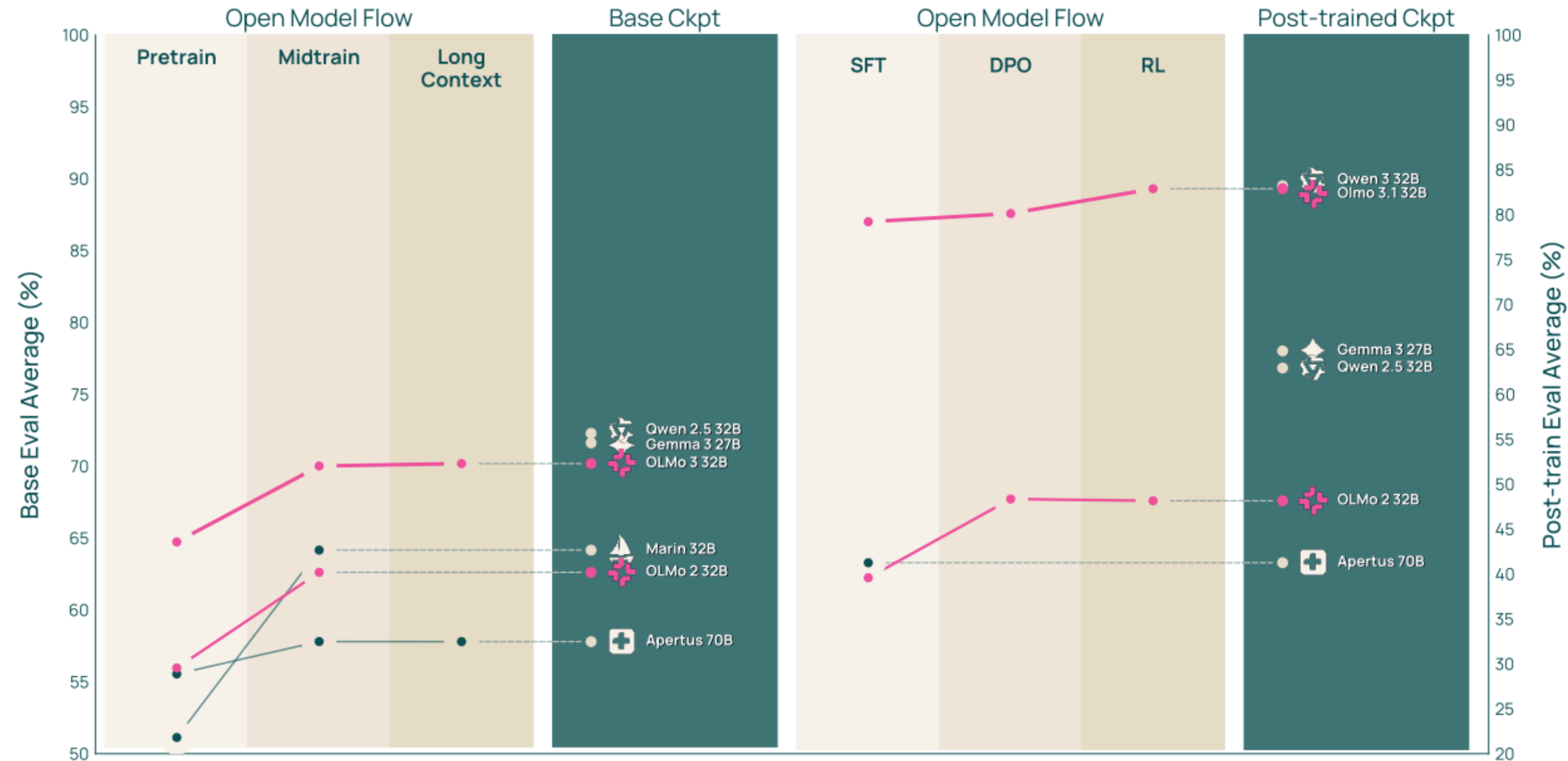


Figure from <https://arxiv.org/pdf/2512.13961> [Olmo 3, 2025]



# Pre-training data

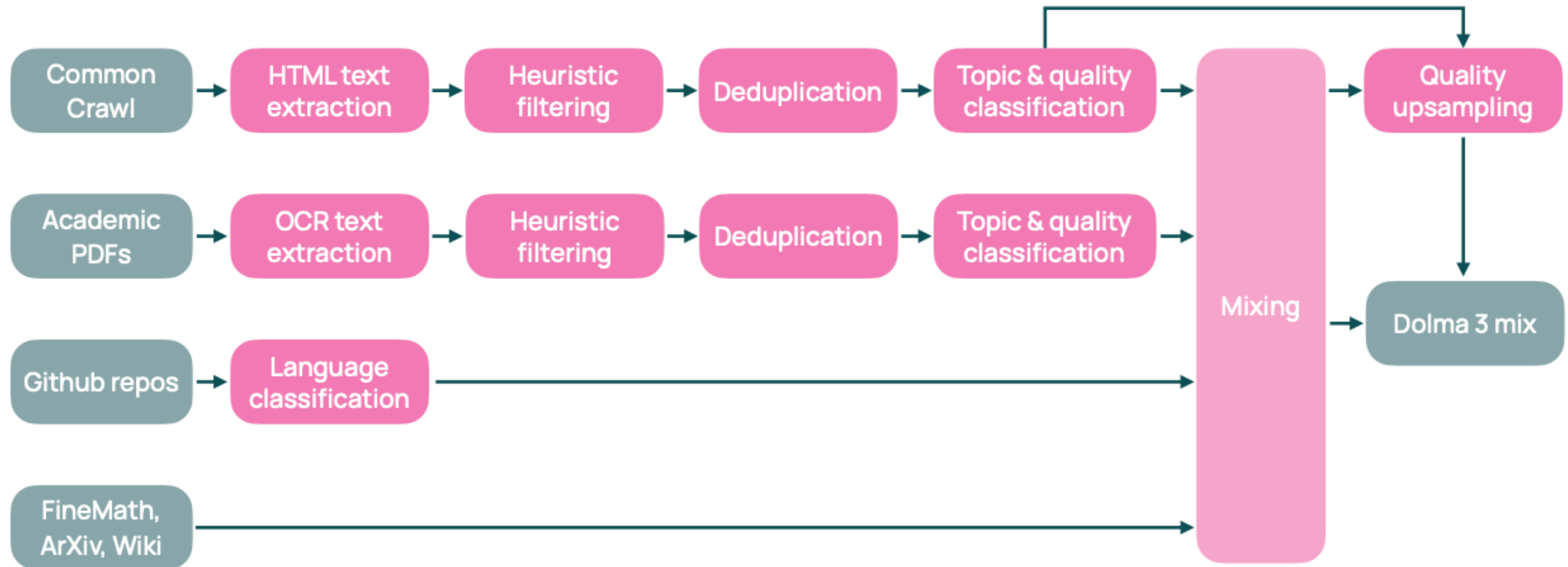
## Training on large amount of web data

| Source                 | Type               | 9T Pool |       | 6T Mix        |       | 150B Mix      |       |
|------------------------|--------------------|---------|-------|---------------|-------|---------------|-------|
|                        |                    | Tokens  | Docs  | Tokens        | Docs  | Tokens        | Docs  |
| Common Crawl           | Web pages          | 8.14T   | 9.67B | 4.51T (76.1%) | 3.15B | 121B (76.9%)  | 84.5M |
| OLMOCR science PDFs    | Academic documents | 972B    | 101M  | 805B (13.6%)  | 83.8M | 19.9B (12.6%) | 2.25M |
| Stack-Edu (Rebalanced) | GitHub code        | 137B    | 167M  | 409B (6.89%)  | 526M  | 11.1B (7.06%) | 14.3M |
| arXiv                  | Papers with LaTeX  | 21.4B   | 3.95M | 50.8B (0.86%) | 9.10M | 1.29B (0.82%) | 247K  |
| FineMath 3+            | Math web pages     | 34.1B   | 21.4M | 152B (2.56%)  | 95.5M | 4.10B (2.60%) | 2.57M |
| Wikipedia & Wikibooks  | Encyclopedic       | 3.69B   | 6.67M | 2.51B (0.04%) | 4.24M | 64.6M (0.04%) | 119K  |
| Total                  |                    | 9.31T   | 9.97B | 5.93T (100%)  | 3.87B | 157B (100%)   | 104M  |

**Table 4 Composition of Dolma 3 Mix** including our 9T pool of data, the 6T mix we used for final model training, and the 150B mix we used for experimentation.

# Pre-training data

Training on large amount of web data



**Figure 8 Data curation flow** for pretraining data sources in DOLMA 3 MIX.



# Pre-training data

## Training on large amount of web data

| Pipeline Step       | Docs Removed (B) | % of pool removed | % of total time |
|---------------------|------------------|-------------------|-----------------|
| URL Filters         | 2.3              | 0.9               | 1.68            |
| Length Filters      | 103.4            | 40.42             | 8.03            |
| Symbol Filters      | 56.5             | 22.1              | 4.13            |
| Internal Repetition | 32.1             | 12.53             | 31.41           |
| Line Modifiers      | 7.1              | 2.79              | 10.0            |
| English Filter      | 6.2              | 2.44              | 14.3            |
| MadLad Filters      | 9.3              | 3.65              | 5.87            |
| Quality Classifiers | 0.0              | 0.0               | 24.58           |

**Table 36 Web data processing step cost and removal breakdown** during the heuristic processing steps. We started with 252.6B documents and ended with 38.7B documents for a total removal rate of 84.9%. This procedure took, in aggregate, approximately 1,030 hours on i4i.32xlarge EC2 instances.



# Why so much data cleaning?

## Effect of de-duplicating

By deduplicating

- Cleaner data leads to more efficient training
- Lower perplexity at less tokens processed

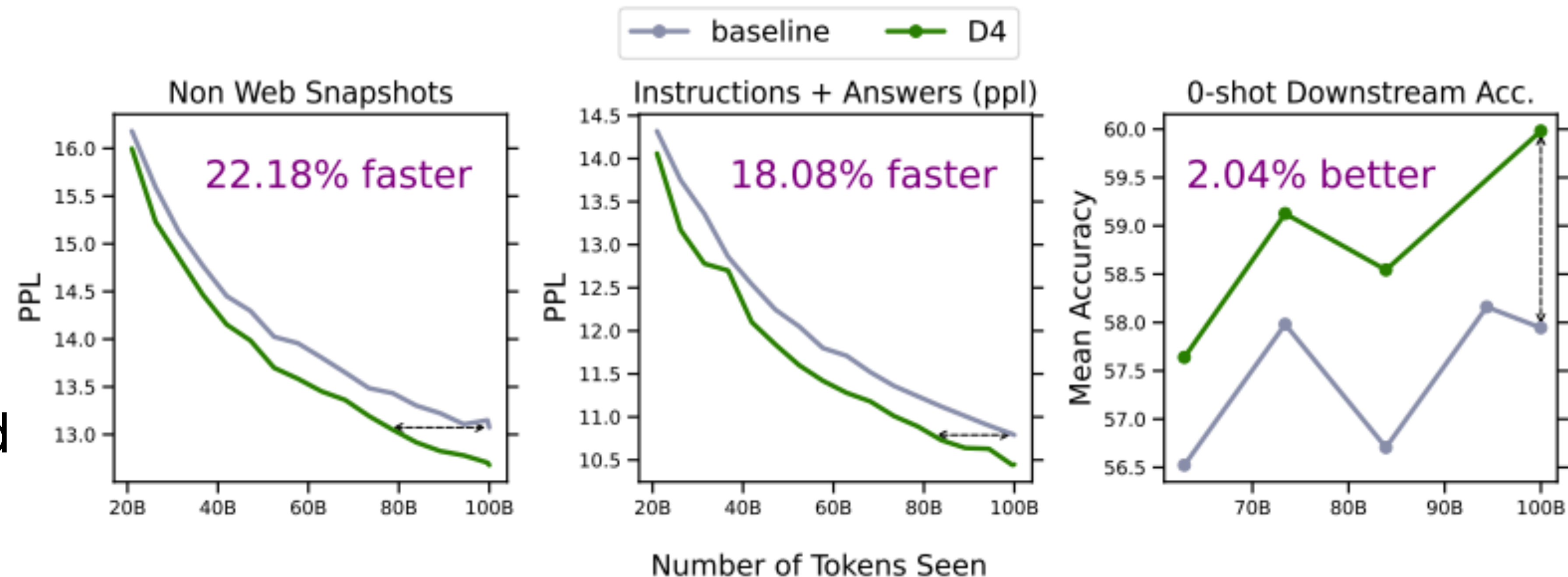


Figure 1: Learning curves for 6.7B OPT model pretraining on 100B tokens, with data selected with D4 (pink line) and randomly (gray line). D4 significantly outperforms baseline training, getting between 18-20% efficiency gains on validation perplexity and 2% increase in average 0-shot downstream accuracy across 16 NLP tasks. See Section A.2 for full learning curves.

# Why so much data cleaning?

## Effect of de-duplicating

D4: Combines SemDeDup and SSL prototypes

Start with dataset D

- Use SemDeDup on D to get D'
- Cluster points in D' using K-means
- Apply SSL prototypes on D'

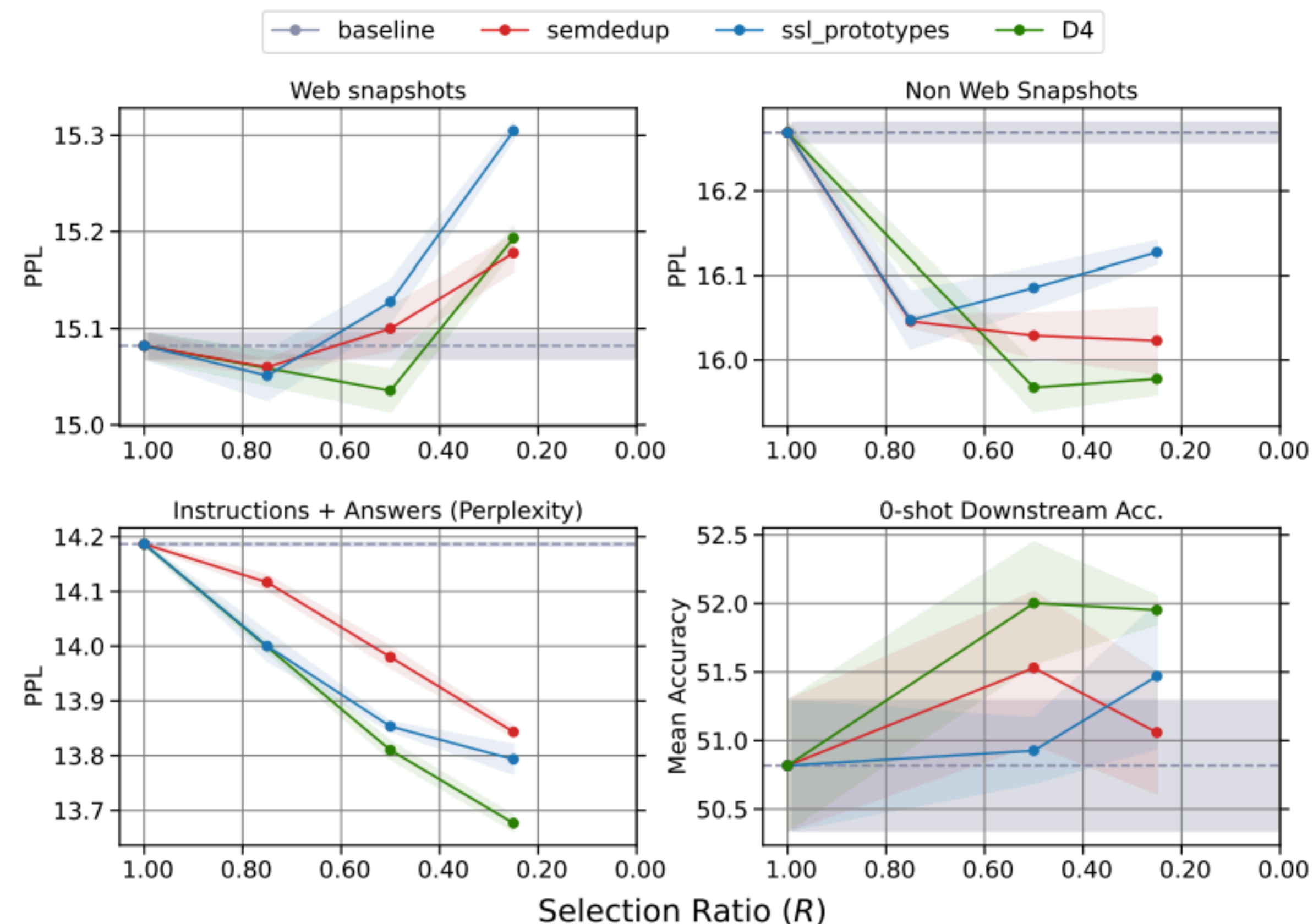
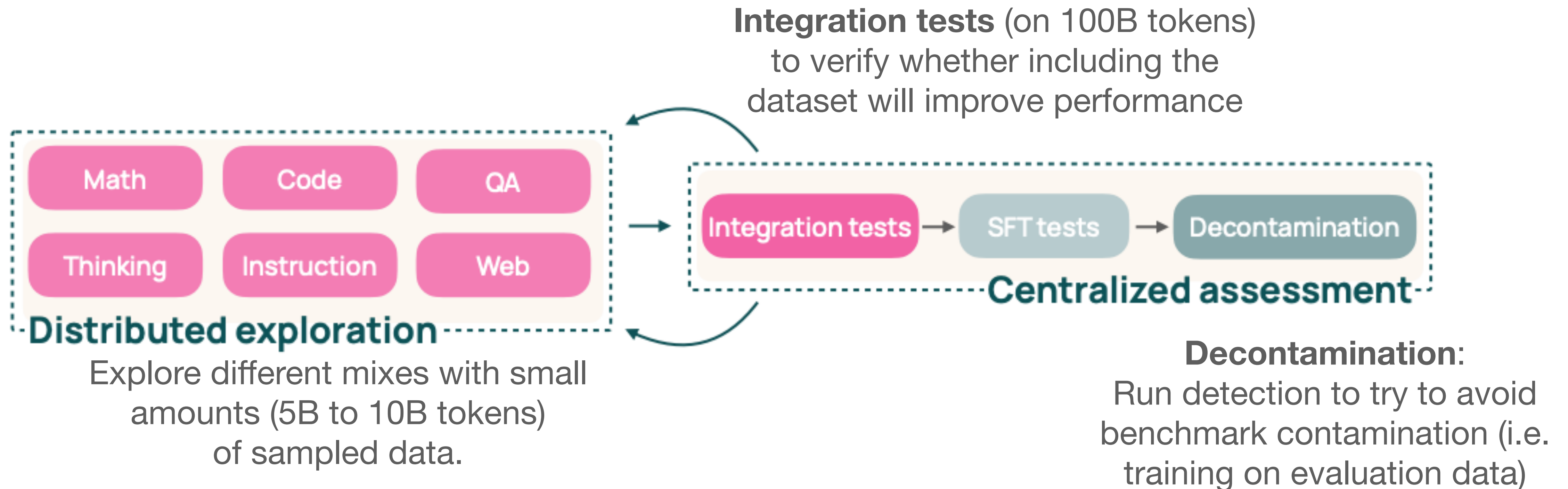


Figure 2: Comparison of data selection methods on validation perplexity. Each point denotes a 1.3B OPT model trained on 40B tokens. The x-axis denotes the selection ratio  $R$ . The y-axis for the top 2 and bottom left graph depicts perplexity; the bottom right graph is average downstream on 16 NLP tasks from Zhang et al. [59]. The grey line denotes the value for baseline training. Shaded error is standard error across 3 seeds. **Each point on this graph is trained on the same token budget:** when we decrease  $R$ , we jointly increase the size of the source dataset (e.g. choosing 1/4 of documents from a 4x'ed sized source dataset).



# Mid-training

## Training on cleaned, specialized data



# Datasets with contamination

| Midtraining Data Sources | Dolmino 1 Flan                | 2e4          | 6e3                | 0       | 0         | 0     | 809  | 124       | 0             | 127       | 10      | 0            | 0             | 35         | 0              | 0                 | 6e3      | 68        | 3e3  | 0        | 0         | 0         | 0                            | 0          | 0              |
|--------------------------|-------------------------------|--------------|--------------------|---------|-----------|-------|------|-----------|---------------|-----------|---------|--------------|---------------|------------|----------------|-------------------|----------|-----------|------|----------|-----------|-----------|------------------------------|------------|----------------|
|                          | Tulu 3 SFT                    | 1e4          | 270                | 0       | 14        | 0     | 85   | 42        | 2             | 25        | 5       | 0            | 0             | 3          | 0              | 0                 | 287      | 6e3       | 893  | 554      | 1e3       | 1e3       | 689                          | 260        | 14             |
|                          | Nemotron Synth QA             | 6e3          | 692                | 10      | 2e3       | 79    | 2    | 23        | 167           | 0         | 80      | 74           | 17            | 15         | 31             | 22                | 519      | 689       | 27   | 1e3      | 41        | 64        | 189                          | 2          | 0              |
|                          | Dolmino Math                  | 4e3          | 0                  | 2e3     | 49        | 1e3   | 0    | 0         | 0             | 0         | 0       | 0            | 0             | 0          | 0              | 0                 | 0        | 0         | 0    | 0        | 2         | 1         | 0                            | 0          | 0              |
|                          | Common Crawl (High Q.)        | 2e3          | 0                  | 0       | 0         | 0     | 0    | 0         | 0             | 0         | 0       | 0            | 0             | 0          | 0              | 0                 | 2e3      | 50        | 0    | 256      | 0         | 1         | 119                          | 0          | 0              |
|                          | StackEdu (FIM)                | 876          | 0                  | 0       | 0         | 0     | 0    | 0         | 0             | 0         | 0       | 0            | 0             | 0          | 0              | 0                 | 792      | 0         | 0    | 24       | 0         | 1         | 58                           | 0          | 1              |
|                          | Gemini Reasoning Traces       | 606          | 0                  | 513     | 31        | 0     | 0    | 0         | 0             | 0         | 0       | 0            | 43            | 0          | 19             | 0                 | 0        | 0         | 0    | 0        | 0         | 0         | 0                            | 0          | 0              |
|                          | OLMOCR Science PDFs (High Q.) | 554          | 0                  | 0       | 0         | 0     | 0    | 0         | 0             | 0         | 0       | 0            | 0             | 0          | 0              | 0                 | 97       | 4         | 1    | 390      | 0         | 19        | 33                           | 10         | 0              |
|                          | Sponge                        | 308          | 0                  | 0       | 0         | 0     | 0    | 0         | 0             | 0         | 0       | 0            | 0             | 0          | 0              | 0                 | 38       | 1         | 0    | 190      | 0         | 0         | 79                           | 0          | 0              |
|                          | General Reasoning Mix         | 113          | 0                  | 6       | 68        | 3     | 0    | 0         | 0             | 0         | 0       | 0            | 5             | 0          | 0              | 0                 | 0        | 0         | 0    | 4        | 0         | 27        | 0                            | 0          | 0              |
|                          |                               | Total contam | Evaluated splits:  |         |           |       |      |           |               |           |         |              | All           |            |                |                   |          |           |      |          |           |           | Occurrences Of Contamination |            |                |
|                          |                               |              | Val/Test           |         |           |       |      |           |               |           |         |              |               |            |                |                   |          |           |      |          |           |           |                              |            |                |
|                          |                               |              | Benchmark (Metric) |         |           |       |      |           |               |           |         |              |               |            |                |                   |          |           |      |          |           |           |                              |            |                |
|                          |                               |              |                    |         |           |       |      |           |               |           |         |              |               |            |                |                   |          |           |      |          |           |           |                              |            |                |
| Perf Δ                   |                               |              | 1.7                | 2.0     | -1.2      | -1.6  | 13.9 | 0.4       | -0.4          | -2.4      | 0.6     | -0.1         | -0.7          | -0.0       | 0.6            | 0.9               | -1.4     | 0.0       | 1.4  | -0.3     | 1.8       | 1.1       | -0.4                         | 1.1        | -0.3           |
| % contam                 |                               |              | 27%                | 50%     | 4%        | 100%  | 9%   | 2%        | 2%            | 2%        | 2%      | 0%           | 5%            | 1%         | 24%            | 3%                | 6%       | 3%        | 2%   | 13%      | 3%        | 2%        | 6%                           | 0%         | 0%             |
|                          |                               |              | SQuAD              | Minerva | MMLU (MC) | GSM8K | DROP | CoQA (MC) | HumEval (@16) | DROP (MC) | LAMBADA | MedMCQA (MC) | MedQA En (MC) | SQuAD (MC) | LeetCode (@16) | M-E-HumEval (@16) | Jeopardy | HellaSwag | CoQA | ARC (MC) | PIQA (MC) | CSQA (MC) | SciQ (MC)                    | Winogrande | SocialIQA (MC) |

## Fix Mix: Remove detected contaminated data to avoid inflated benchmark results

Figure from <https://arxiv.org/pdf/2512.13961> [Olmo 3, 2025]

### Benchmark (Metric)



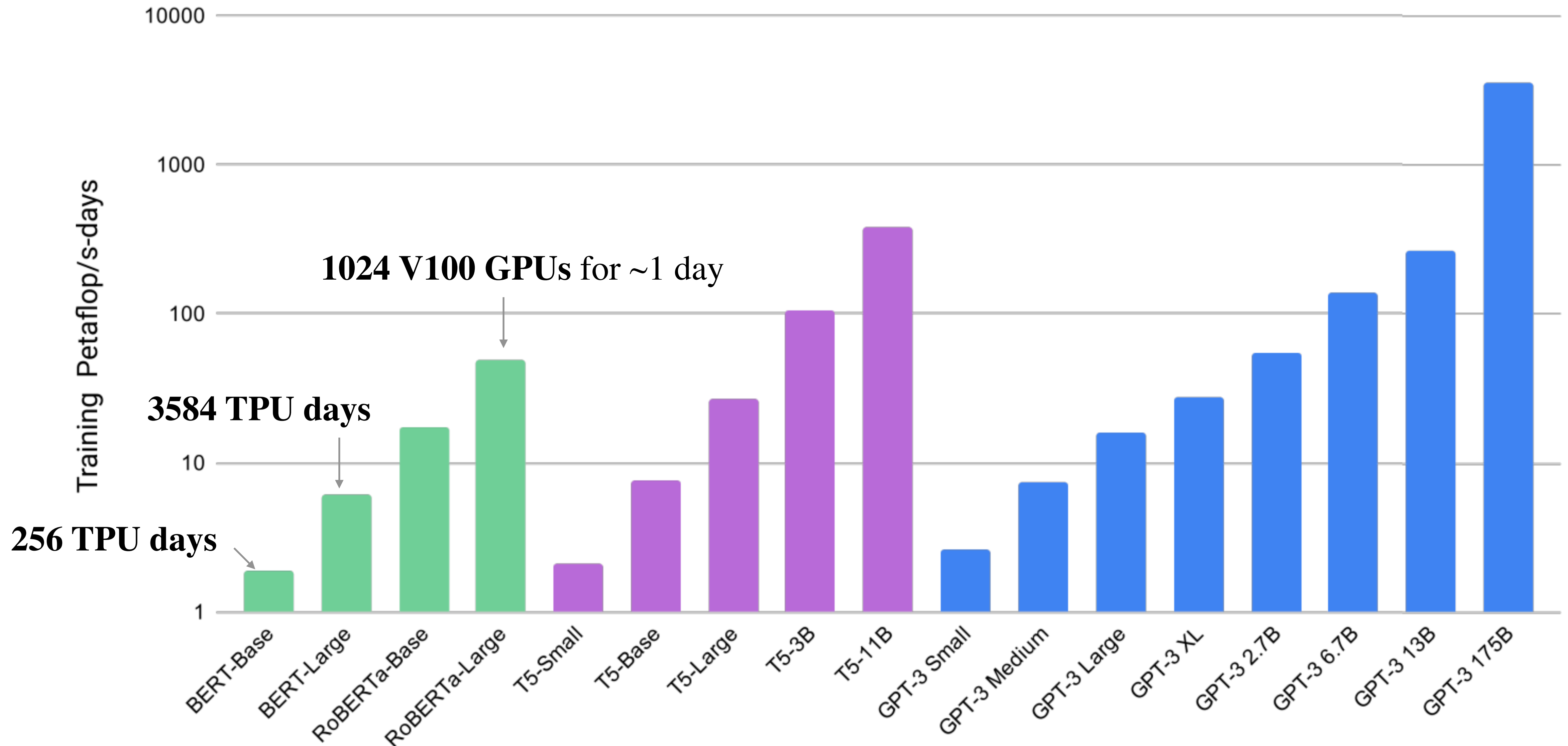
# Mid-training

## Training on long-context data

| Source                         | Length bucket | 600B Pool     |       | 50B Mix       |       |
|--------------------------------|---------------|---------------|-------|---------------|-------|
|                                |               | Tokens        | Docs  | Tokens        | Docs  |
| olmOCR PDFs                    | 8K-16K        | 144B (22.5%)  | 12.7M | 2.27B (4.55%) | 235K  |
| olmOCR PDFs                    | 16K-32K       | 115B (18.0%)  | 5.06M | 1.85B (3.70%) | 110K  |
| olmOCR PDFs                    | 32K-64K       | 106B (16.6%)  | 2.30M | 4.81B (9.63%) | 177K  |
| olmOCR PDFs                    | 64K-128K      | 96.0B (15.0%) | 1.05M | –             | –     |
| olmOCR PDFs                    | 128K-256K     | 60.8B (9.5%)  | 342K  | –             | –     |
| olmOCR PDFs                    | 256K-512K     | 35.1B (5.49%) | 97.1K | –             | –     |
| olmOCR PDFs                    | 512K-1M       | 21.5B (3.36%) | 30.2K | –             | –     |
| olmOCR PDFs                    | 1M+           | 26.9B (4.21%) | 12.2K | –             | –     |
| olmOCR PDFs + synth <b>CWE</b> | 32K-64K       | 8.77B (1.37%) | 189K  | 1.94B (3.88%) | 71.3K |
| olmOCR PDFs + synth <b>REX</b> | 32K-64K       | 24.1B (3.77%) | 492K  | 6.08B (12.2%) | 217K  |
| Midtraining data mix           | Variable      | –             | –     | 33.0B (66.1%) | 79.2M |
| Total                          |               | 639B          | 22.3M | 50.0B (100%)  | 80.0M |

**Table 11 Composition of Dolma 3 Longmino Mix.** The 100B mix for OLMO 3 32B maintains the same proportions as the 50B mix. Length buckets are reported in DOLMA 3 tokens.

## Total Compute Used During Training



**How does LLM performance scale  
as we increase model and data size?**



---

# Scaling Laws for Neural Language Models

---

**Jared Kaplan \***

Johns Hopkins University, OpenAI

jaredk@jhu.edu

**Sam McCandlish\***

OpenAI

sam@openai.com

**Tom Henighan**

OpenAI

henighan@openai.com

**Tom B. Brown**

OpenAI

tom@openai.com

**Benjamin Chess**

OpenAI

bchess@openai.com

**Rewon Child**

OpenAI

rewon@openai.com

**Scott Gray**

OpenAI

scott@openai.com

**Alec Radford**

OpenAI

alec@openai.com

**Jeffrey Wu**

OpenAI

jeffwu@openai.com

**Dario Amodei**

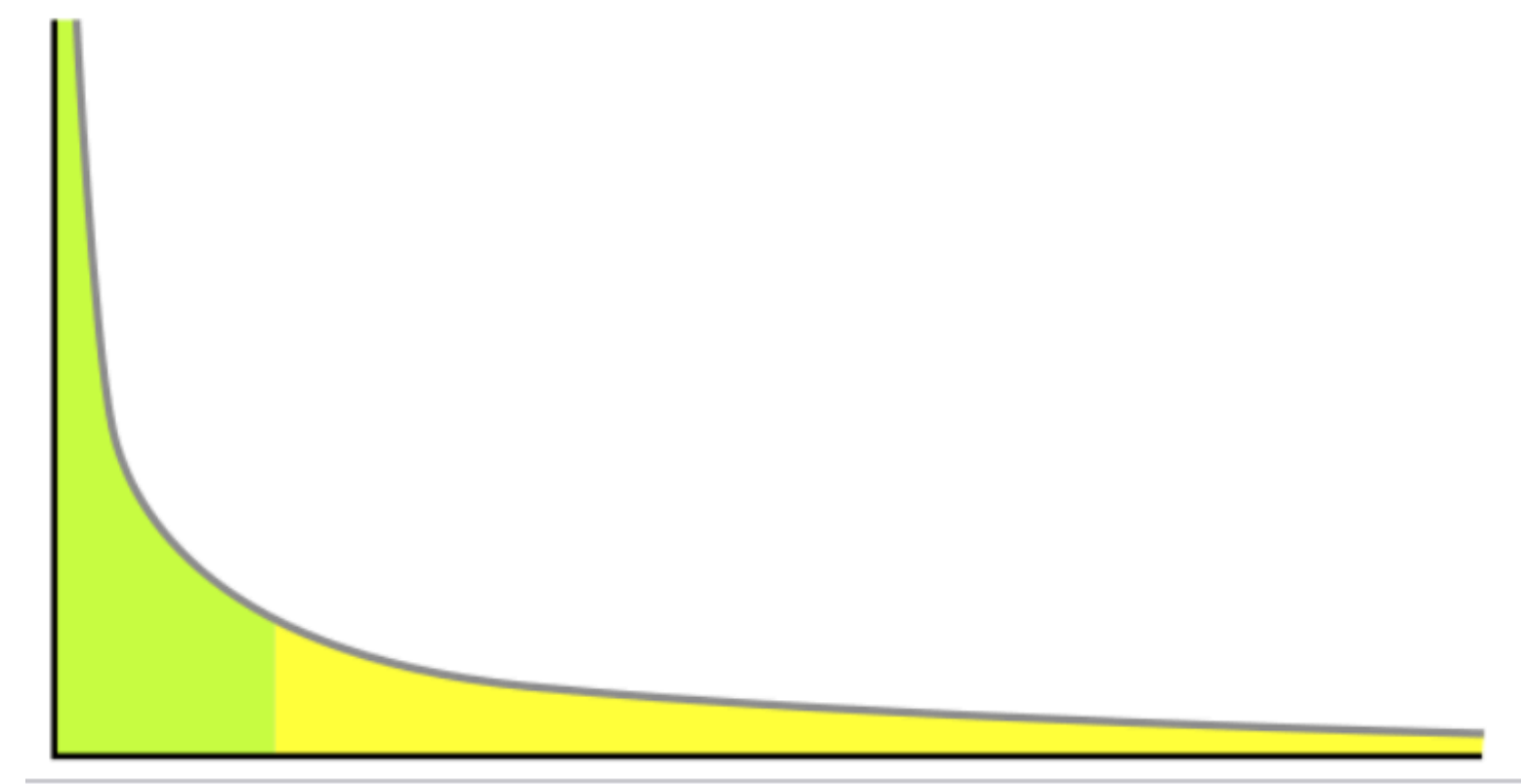
OpenAI

damodei@openai.com



# Scaling Laws for LLMs

## Power laws



For LLMs, we are interested in how the test performance scales with relation to

- Model size: number of model parameters **N** (excluding subword embeddings)
- Data size: number of tokens trained on **D**
- Amount of compute (MFLOPs) **C** (1 PetaFLOP-day (PF-day) is  $8.64 \times 10^{19}$  FLOPS)

## Findings

- Model performance scales as **power law** of model size and data size
- Power law: relation between two quantities where one quantity increases as a power of another
  - $f(x) = (a/x)^k$  e.g. model performance vs. model size
- N, D, C are dominant. Other choices in hyperparameters like width vs. depth are less relevant

<https://openai.com/research/ai-and-compute>

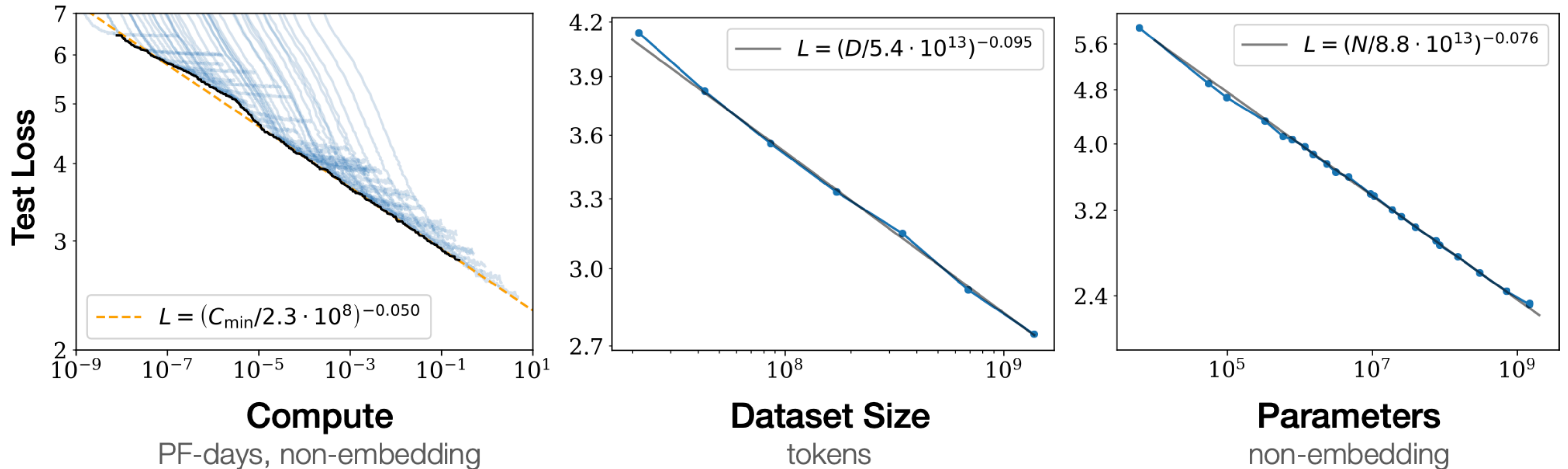
# Model size: computing the number of parameters

| Operation                    | Parameters                                                                  | FLOPs per Token                                                              |
|------------------------------|-----------------------------------------------------------------------------|------------------------------------------------------------------------------|
| Embed                        | $(n_{\text{vocab}} + n_{\text{ctx}}) d_{\text{model}}$                      | $4d_{\text{model}}$                                                          |
| Attention: QKV               | $n_{\text{layer}} d_{\text{model}} 3d_{\text{attn}}$                        | $2n_{\text{layer}} d_{\text{model}} 3d_{\text{attn}}$                        |
| Attention: Mask              | —                                                                           | $2n_{\text{layer}} n_{\text{ctx}} d_{\text{attn}}$                           |
| Attention: Project           | $n_{\text{layer}} d_{\text{attn}} d_{\text{model}}$                         | $2n_{\text{layer}} d_{\text{attn}} d_{\text{embd}}$                          |
| Feedforward                  | $n_{\text{layer}} 2d_{\text{model}} d_{\text{ff}}$                          | $2n_{\text{layer}} 2d_{\text{model}} d_{\text{ff}}$                          |
| De-embed                     | —                                                                           | $2d_{\text{model}} n_{\text{vocab}}$                                         |
| <b>Total (Non-Embedding)</b> | $N = 2d_{\text{model}} n_{\text{layer}} (2d_{\text{attn}} + d_{\text{ff}})$ | $C_{\text{forward}} = 2N + 2n_{\text{layer}} n_{\text{ctx}} d_{\text{attn}}$ |

**Table 1** Parameter counts and compute (forward pass) estimates for a Transformer model. Sub-leading terms such as nonlinearities, biases, and layer normalization are omitted.

# Test performance

- Power law relationship to compute, dataset size, and number of parameters

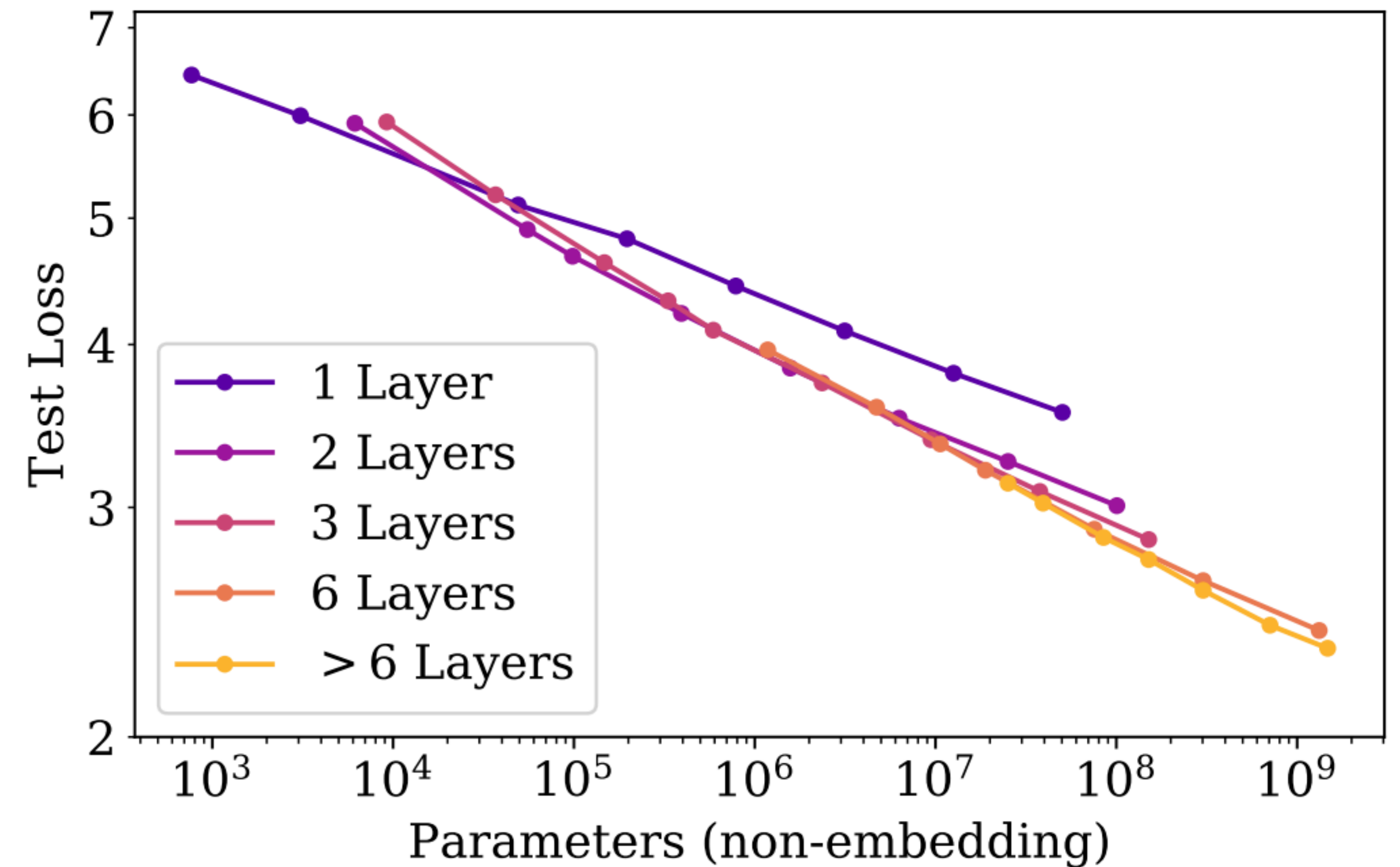
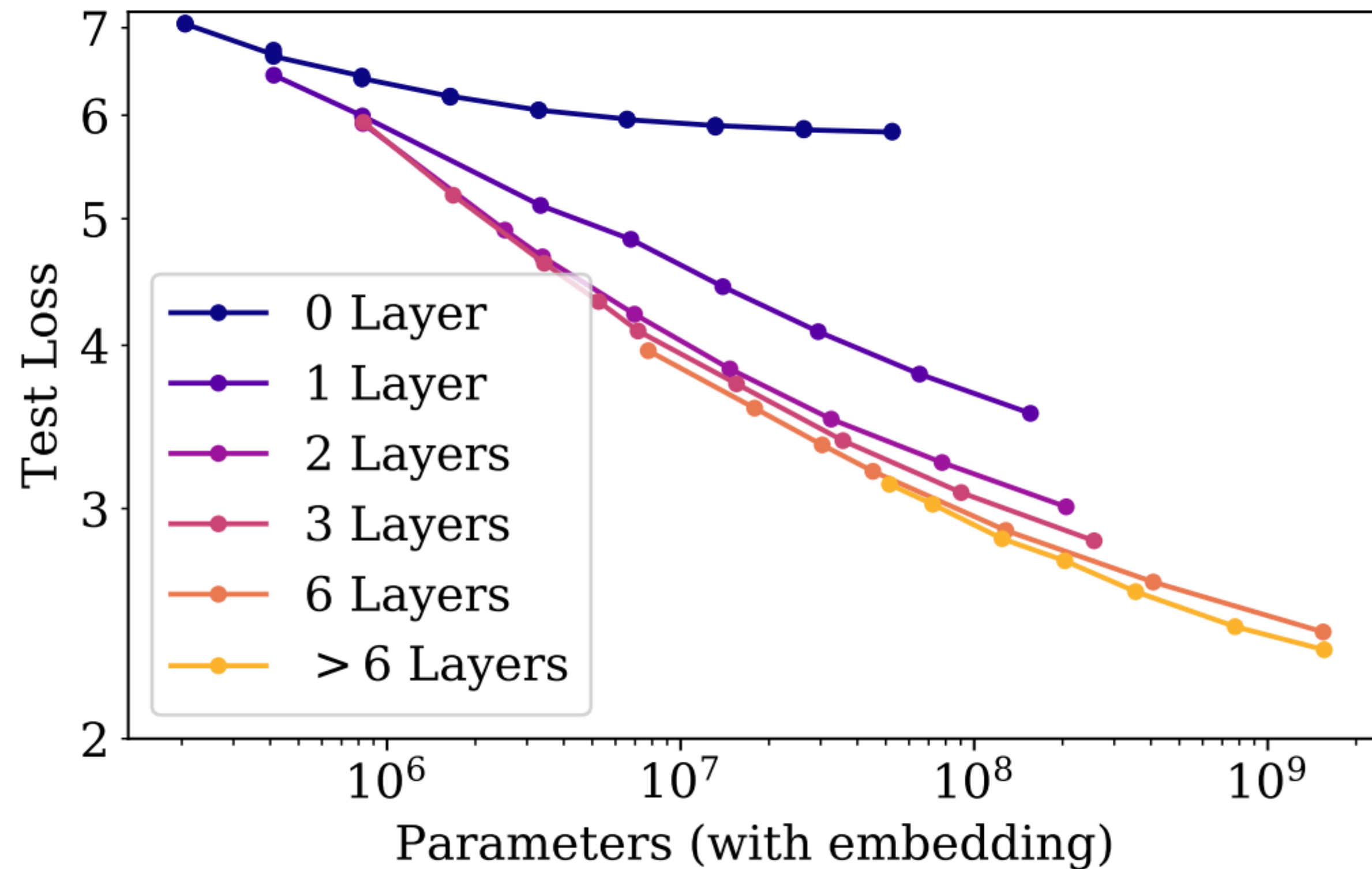


**Figure 1** Language modeling performance improves smoothly as we increase the model size, dataset size, and amount of compute<sup>2</sup> used for training. For optimal performance all three factors must be scaled up in tandem. Empirical performance has a power-law relationship with each individual factor when not bottlenecked by the other two.



# Excluding embeddings from parameter count

- Power law relationship not so clear when embeddings are included





# Power laws for test loss

- Let  $L(\cdot)$  represent the test loss dependent on either parameters  $N$ , or dataset size  $D$  or compute  $C$
- For models with limited number of parameters:

$$L(N) = (N_c/N)^{\alpha_N} \quad \alpha_N \approx 0.076, \quad N_c \approx 8.8 \times 10^{13} \text{ (non-embd params)}$$

- For models with limited dataset size:

$$L(D) = (D_c/D)^{\alpha_D} \quad \alpha_D \approx 0.095, \quad D_c \approx 5.4 \times 10^{13} \text{ (tokens)}$$

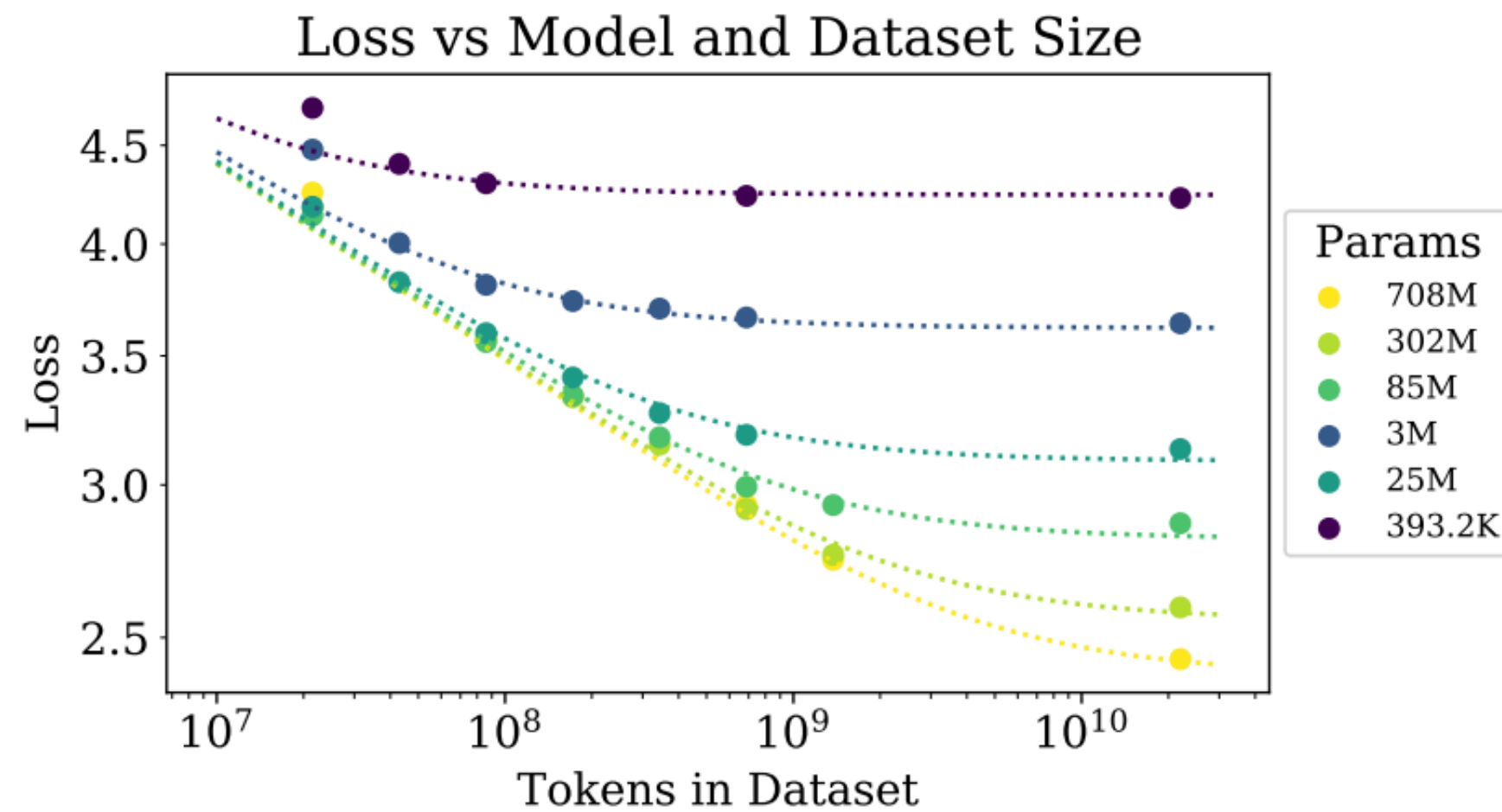
- For models trained with limited compute:

$$L(C) = (C_c^{min}/C_{min})^{\alpha_C^{min}} \quad \alpha_C^{min} \approx 0.050, \quad C_c^{min} \approx 3.1 \times 10^8 \text{ (PF-days)}$$

# Scaling laws for LLMs

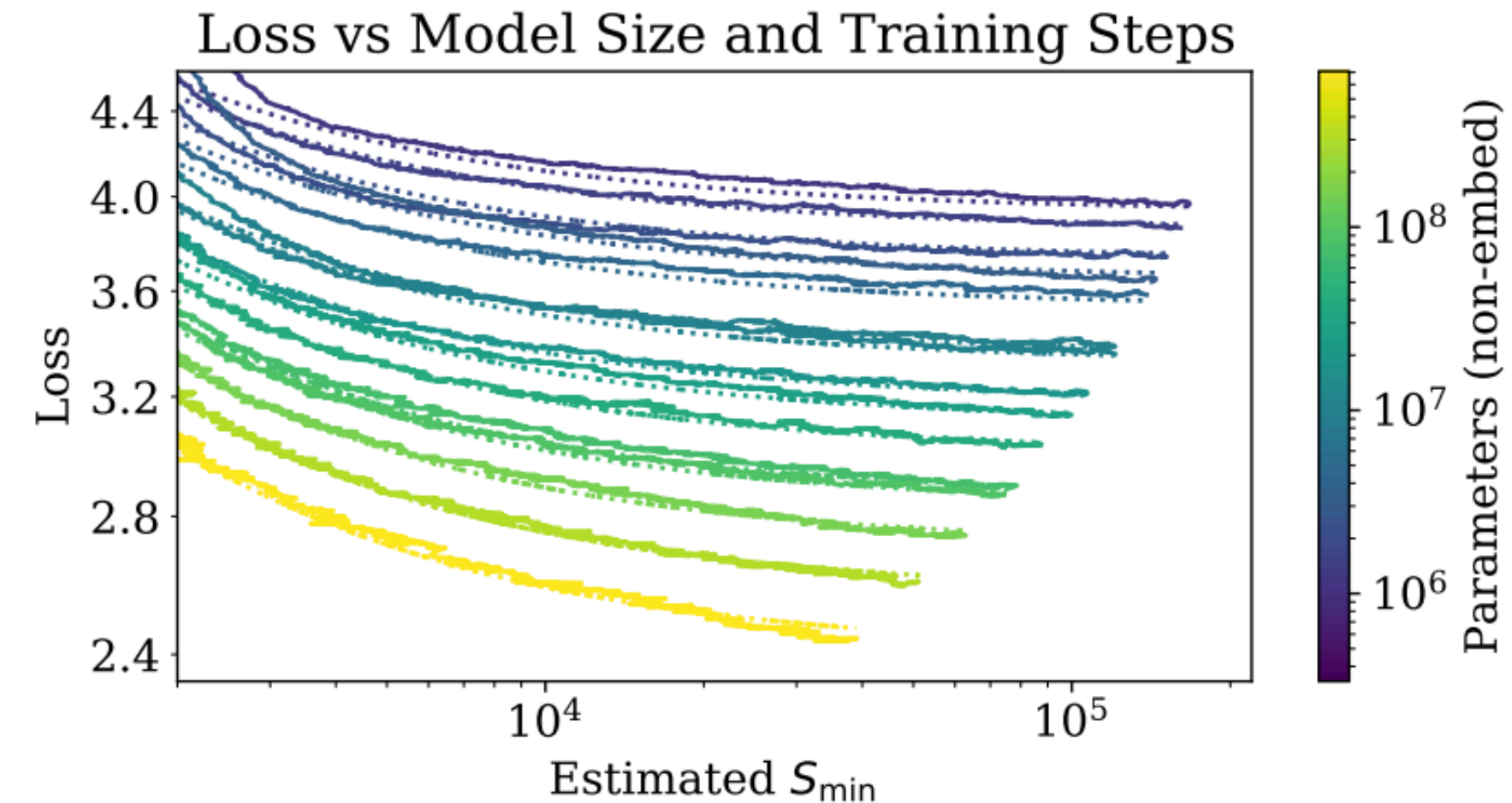
Test loss  $L$  as function  
of model size  $N$  and dataset size  $D$

$$L(N, D) = \left[ \left( \frac{N_c}{N} \right)^{\frac{\alpha_N}{\alpha_D}} + \frac{D_c}{D} \right]^{\alpha_D}$$



Test loss  $L$  after transient period as function  
of model size  $N$  and number of update steps  $S$

$$L(N, S) = \left( \frac{N_c}{N} \right)^{\alpha_N} + \left( \frac{S_c}{S_{\min}(S)} \right)^{\alpha_S}$$

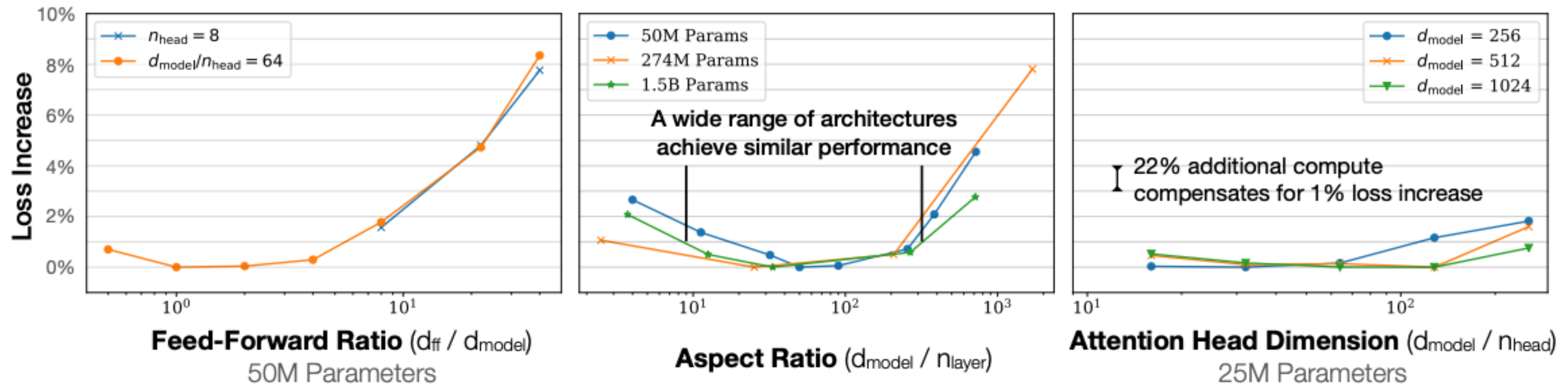


$N$  is number of model parameters (not including vocabulary and positional embeddings)

$D$  is the number of tokens

# Scaling laws for LLMs

- Keeping model size  $N$  fixed, architecture shape doesn't matter that much

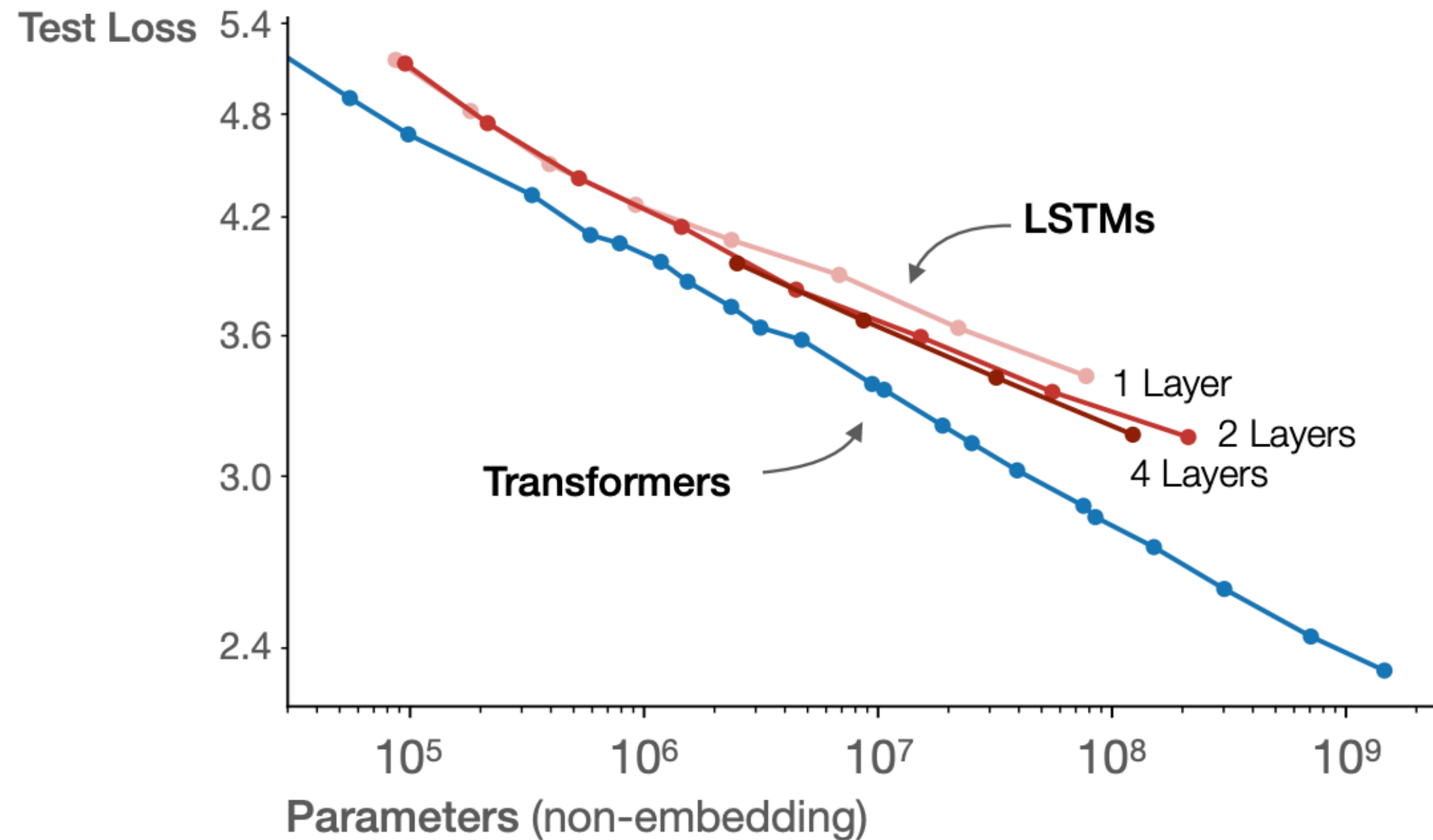




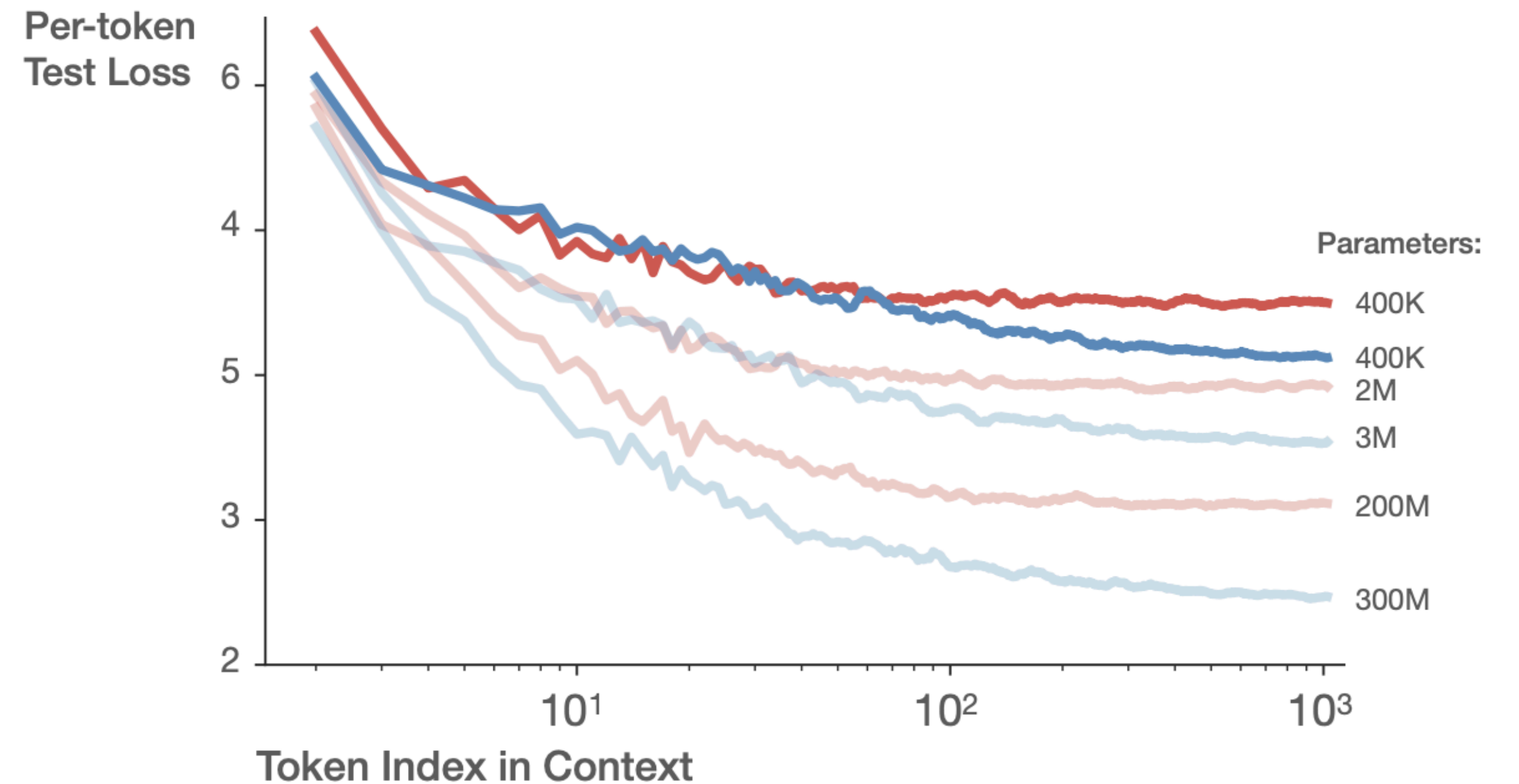
# Comparing LSTM vs Transformers

- **LSTM** cannot take advantage of long context (>100 tokens)

**Transformers asymptotically outperform LSTMs**  
due to improved use of long contexts



**LSTM** plateaus after <100 tokens  
**Transformer** improves through the whole context

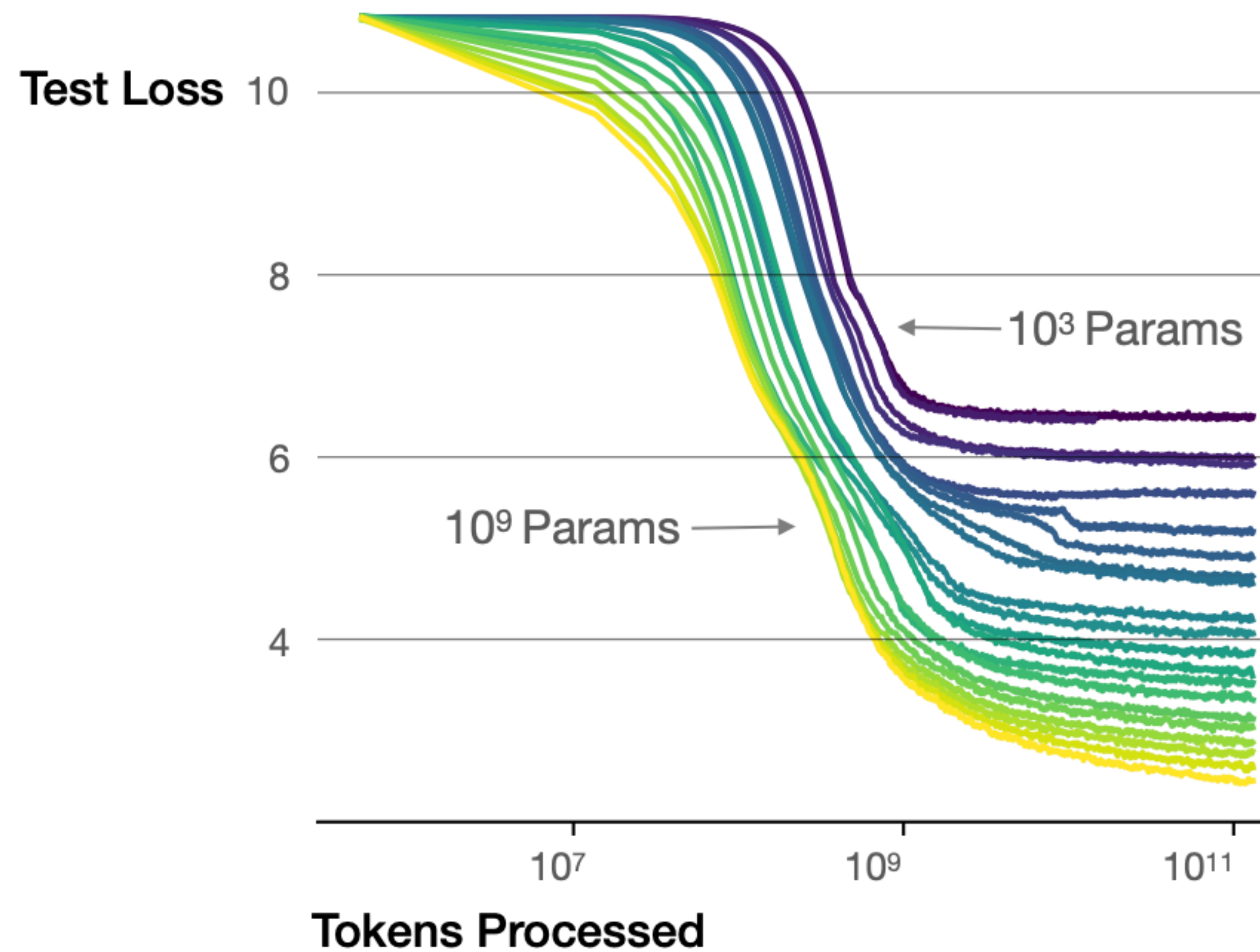




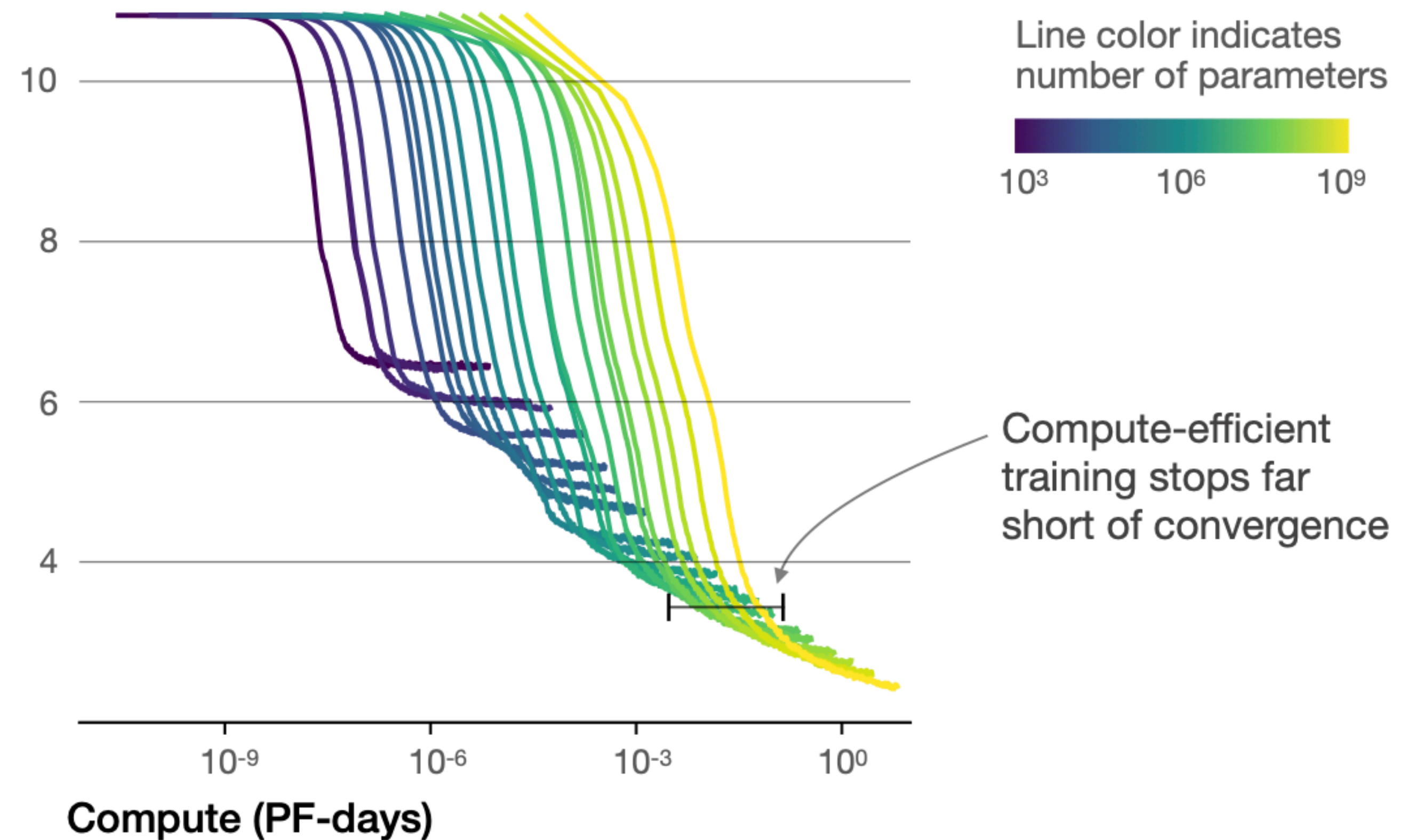
Given a compute budget, what size model  
and amount of data should we train on?

# Large models are more sample-efficient than small models

Larger models require **fewer samples** to reach the same performance



The optimal model size grows smoothly with the loss target and compute budget

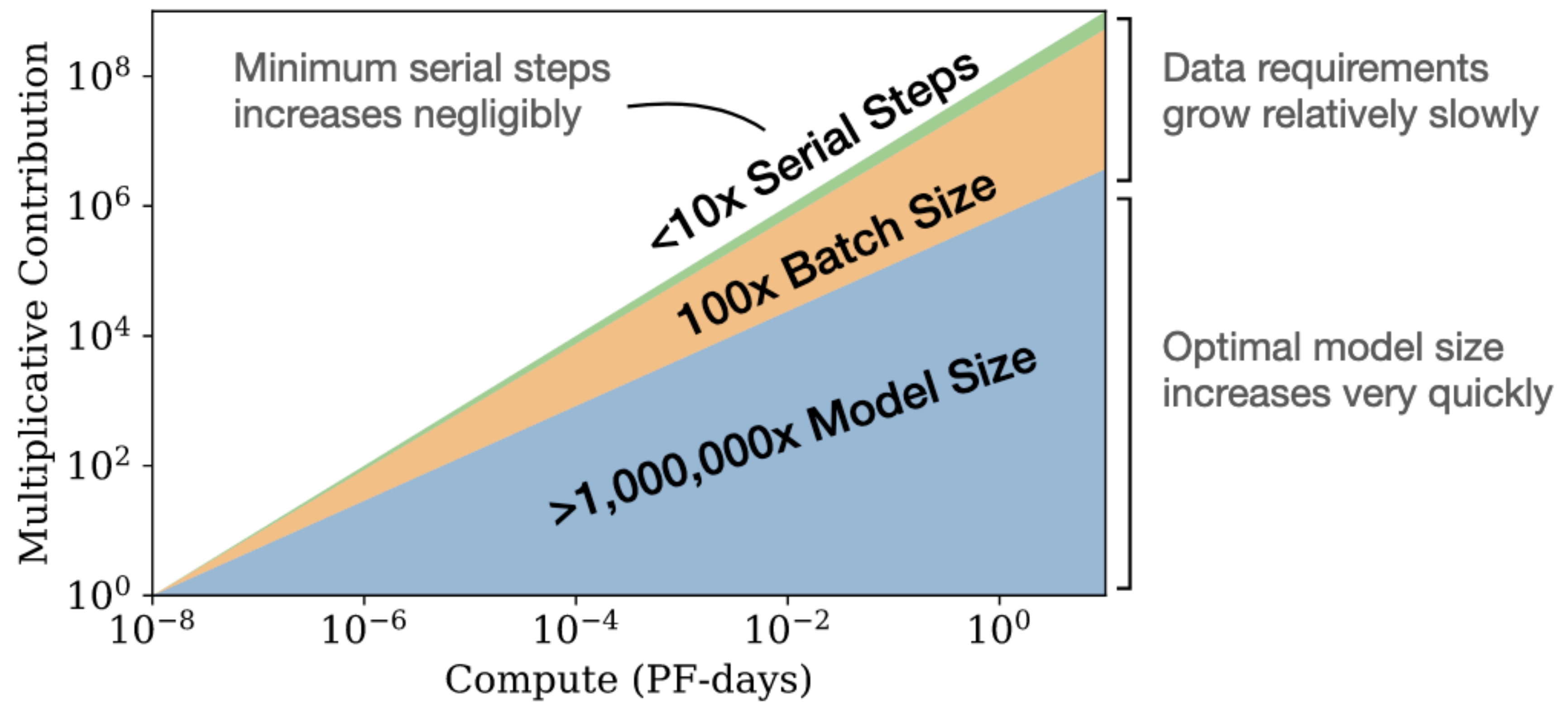


**Figure 2** We show a series of language model training runs, with models ranging in size from  $10^3$  to  $10^9$  parameters (excluding embeddings).

# How to allocate increasing compute?

For compute-efficient training

- Increase **model size** more than data (increase data sublinearly).
- Increase **batch size** as data size increases

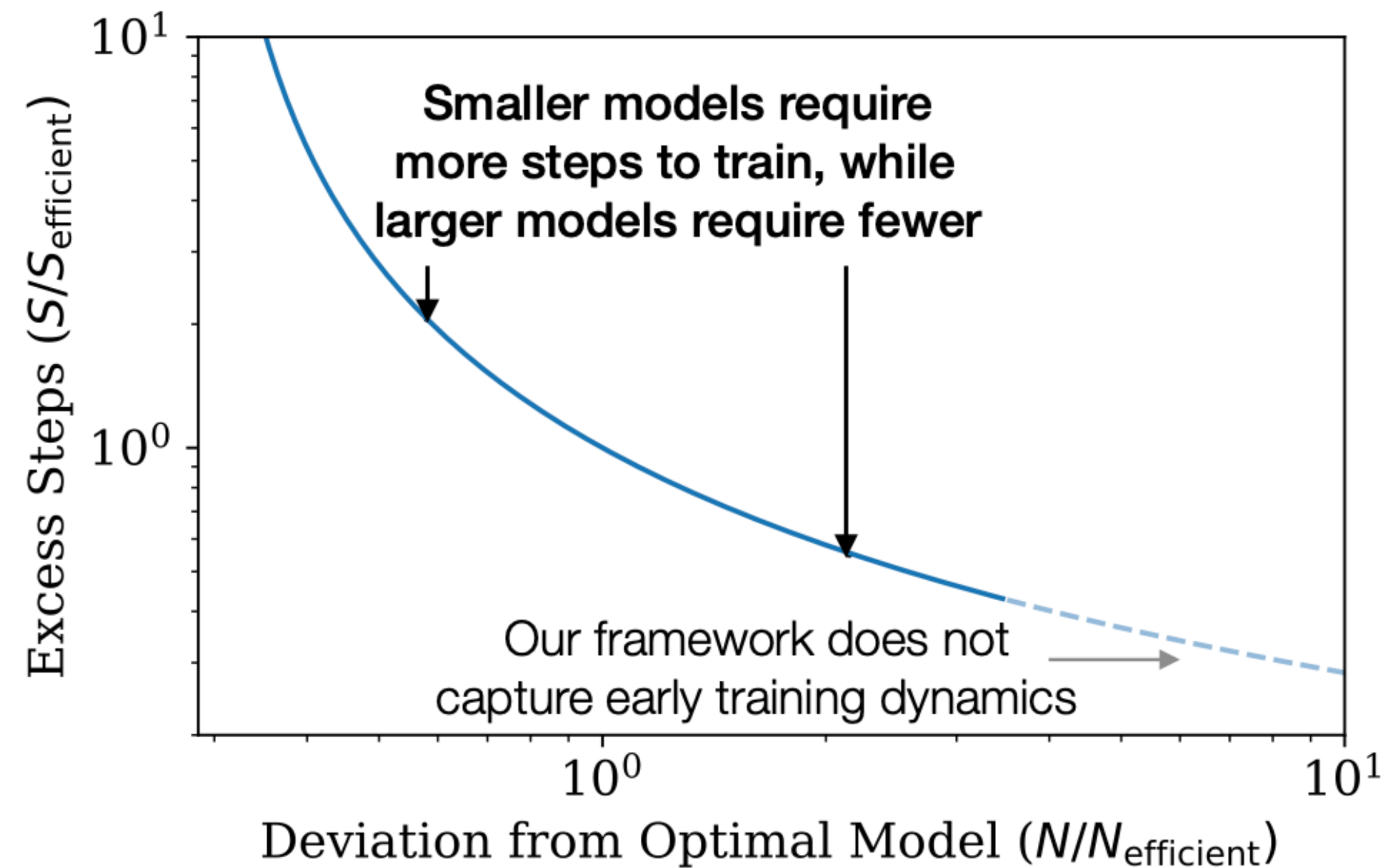
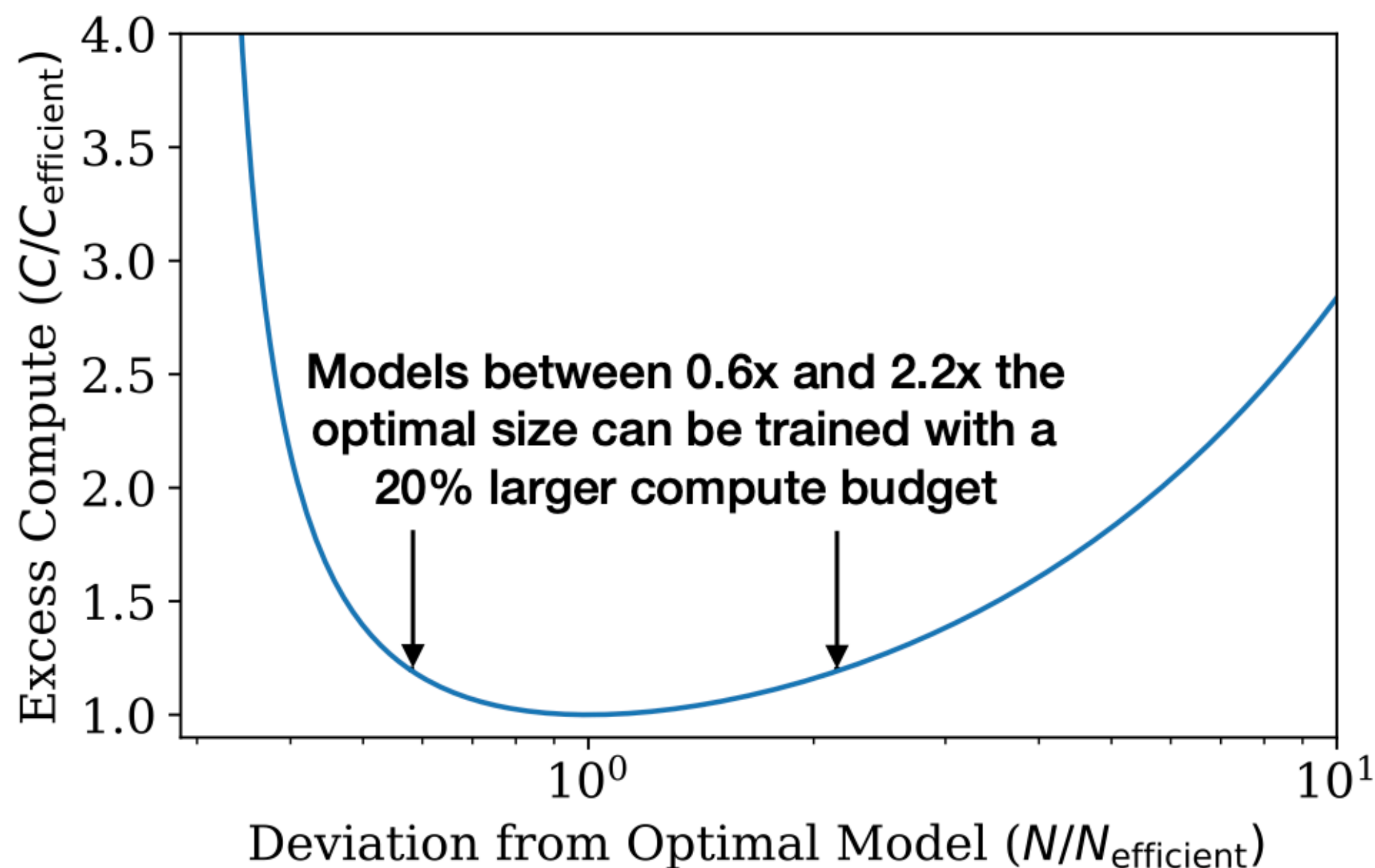


Billion-fold ( $10^9$ ) increase in compute time



# Optimal Allocation of Compute Budget

Training at fixed batch size (should increase batch size with more data)



Models larger than the optimal-size can train faster (with less steps)

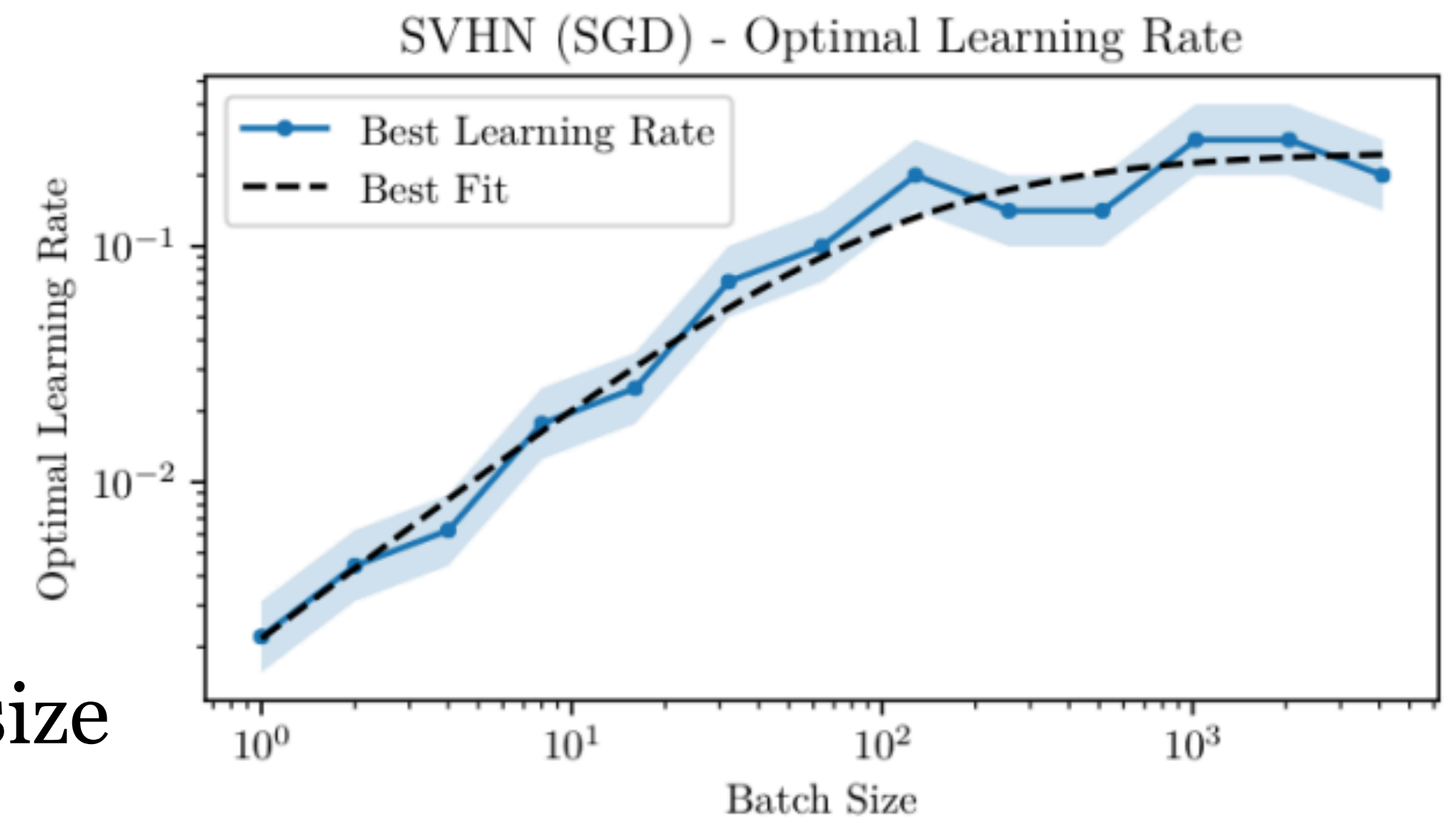
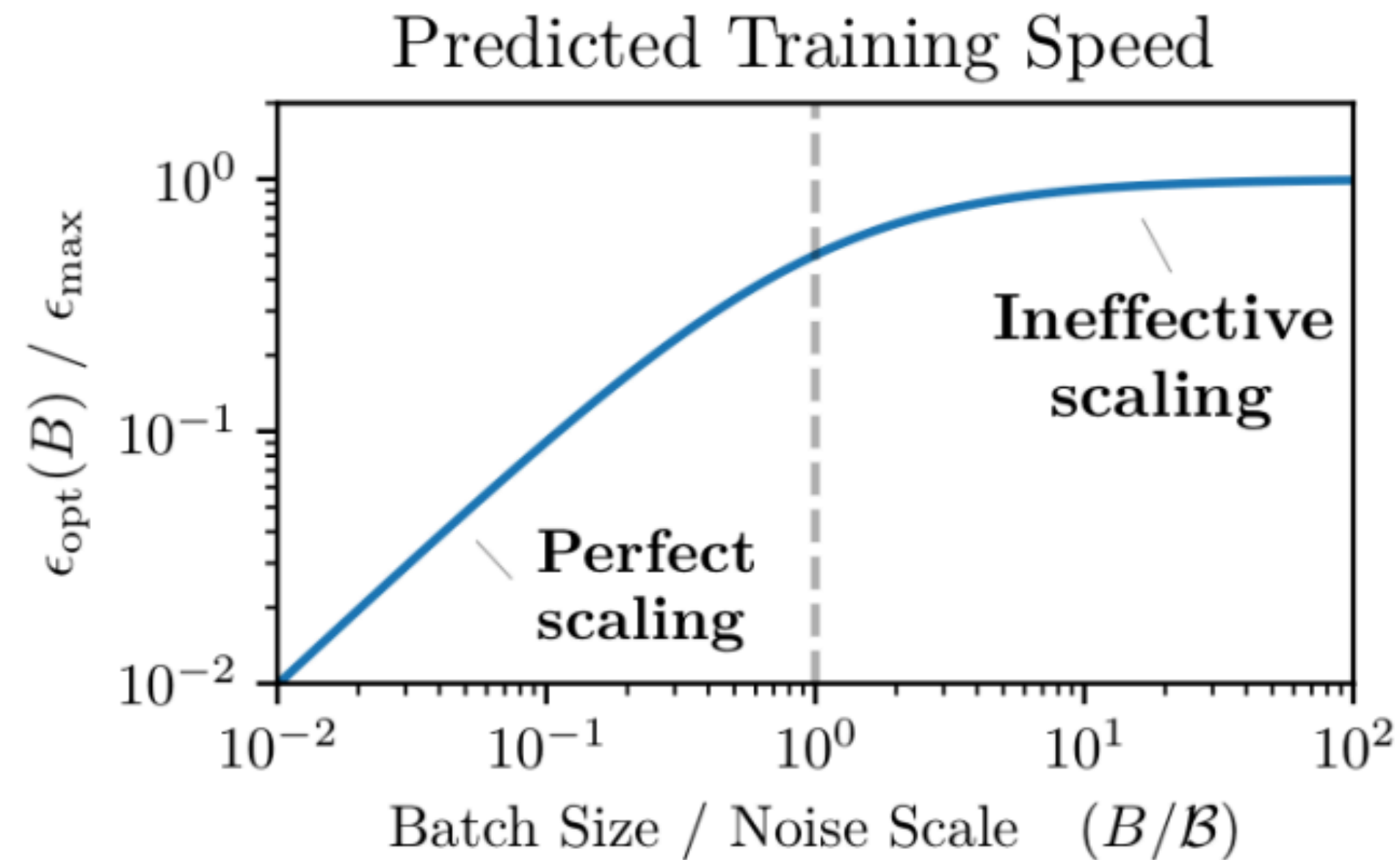
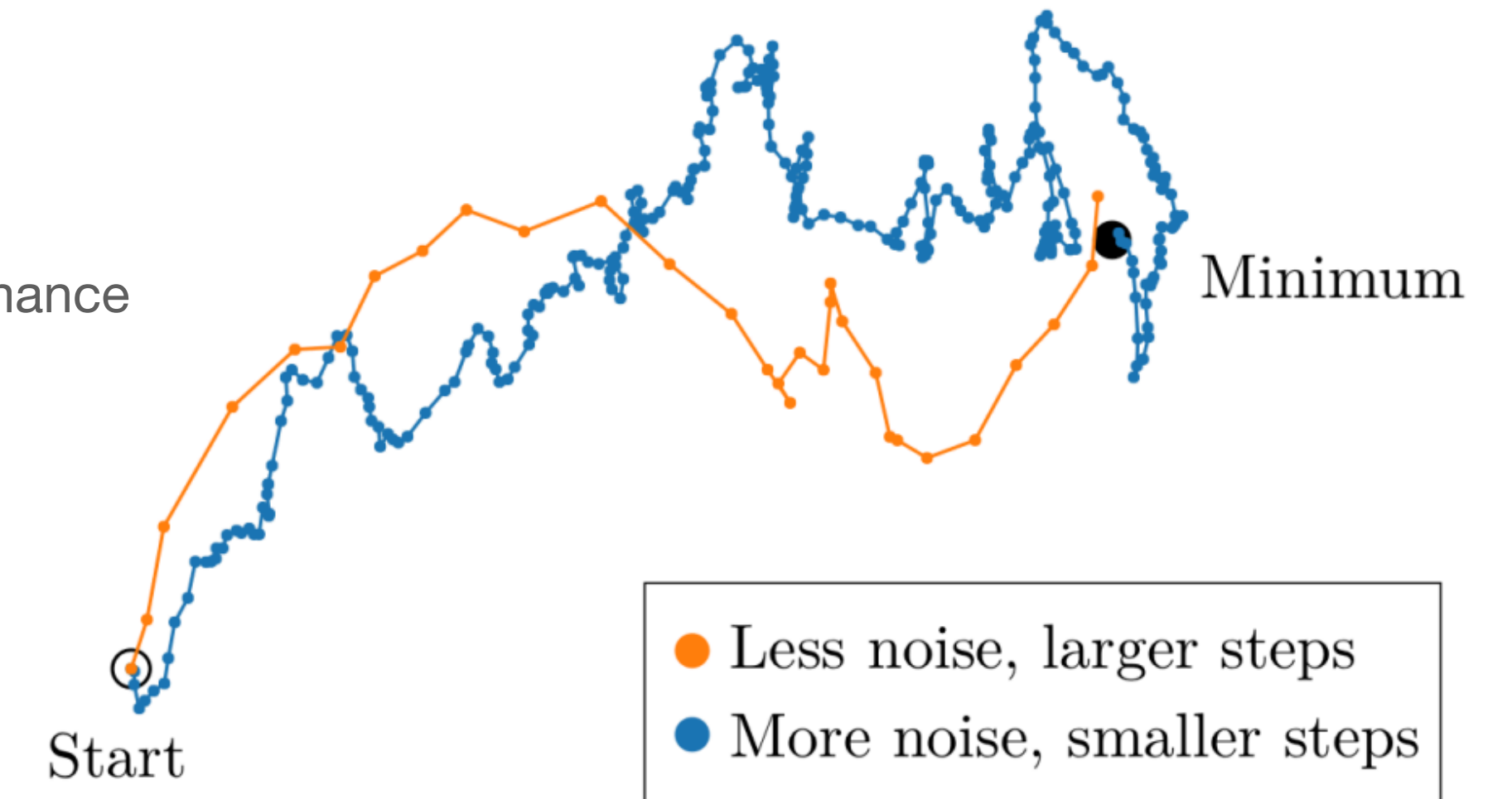


# Critical batch size

For compute efficient training, train with  $B_{\text{crit}} = \frac{E_{\text{min}}}{S_{\text{min}}}$  to reach a given performance

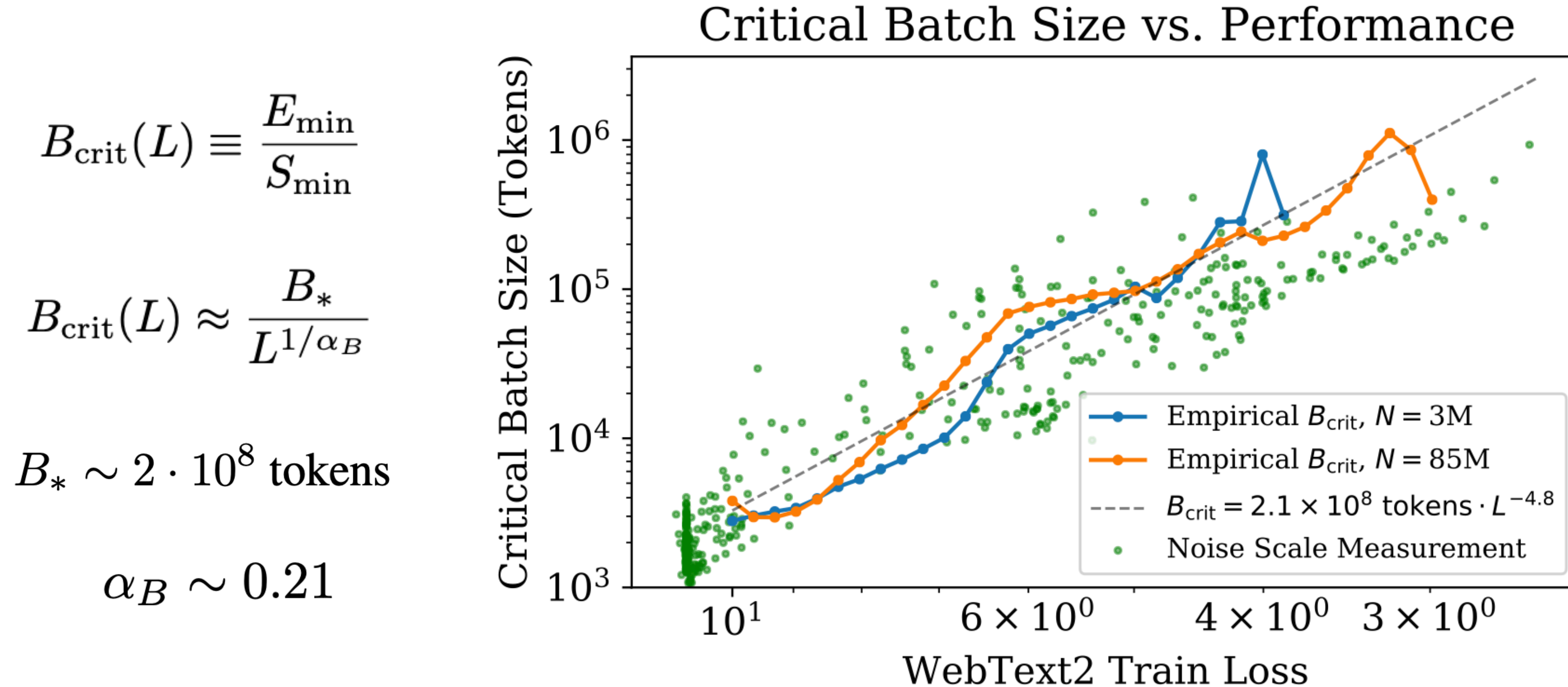
Number of training examples  
Number of training steps

- Larger B: more stable gradient, less training steps
- Critical batch size: above which scaling efficiency decreases significantly



- Optimal learning rate scales linearly with batch size

# Critical batch size as function of test loss



**Figure 10** The critical batch size  $B_{\text{crit}}$  follows a power law in the loss as performance increase, and does not depend directly on the model size. We find that the critical batch size approximately doubles for every 13% decrease in loss.  $B_{\text{crit}}$  is measured empirically from the data shown in Figure 18, but it is also roughly predicted by the gradient noise scale, as in [MKAT18].

# Lessons from scaling LLMs

- Number of model parameters
- Size of dataset  $D$
- Amount of compute (MFLOPs)  $C$
- Performance depends **strongly on scale, weakly on model shape**
- Performance has a **power-law** relationship with each of the three scale factors  $N$ ,  $D$ ,  $C$  when not bottlenecked by the other two
- Performance improves predictably as long as we **scale up  $N$  and  $D$  in tandem**
- Training curves follow predictable power-laws whose parameters are roughly independent of the model size



# Lessons from scaling LLMs

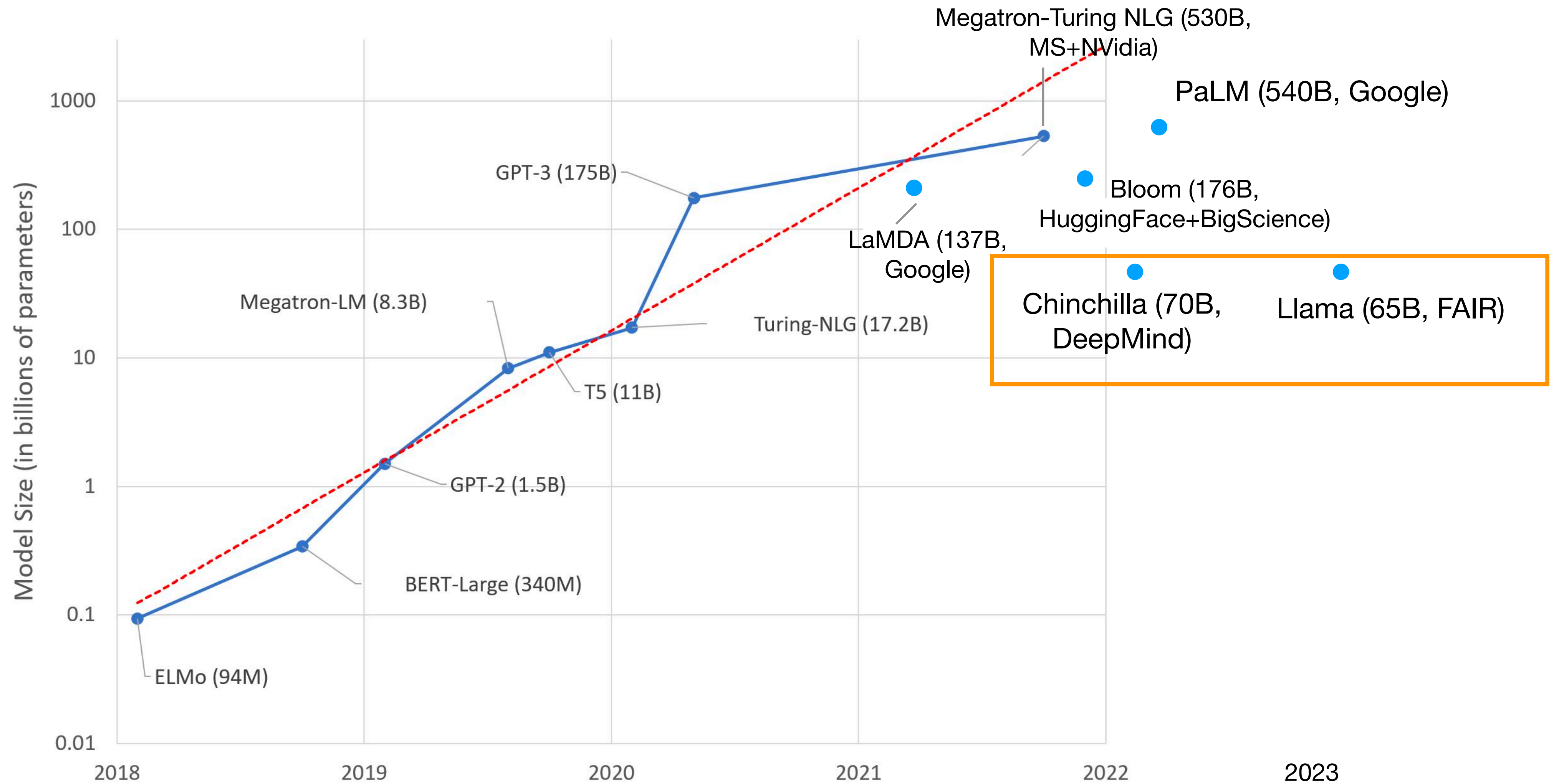
- Transfer to a different distribution incurs a constant penalty but otherwise improves roughly in line with performance on the training set.
- Large models are more sample-efficient than small models, reaching the same level of performance with fewer optimization steps and using fewer data points
- The ideal batch size for training these models is roughly a power of the loss only, and continues to be determinable by measuring the gradient noise scale
- If no constraints on data and model size, with given compute budget  $C$

$$N \propto C^{\alpha_C^{\min}} / \alpha_N \quad B \propto C^{\alpha_C^{\min}} / \alpha_B \quad S \propto C^{\alpha_C^{\min}} / \alpha_S \quad D = B \cdot S$$



Is larger models always better?  
Can we train high-performance smaller  
models with more data?

# Is bigger always better?



<https://huggingface.co/blog/large-language-models>

# Training Compute-Optimal Large Language Models

Jordan Hoffmann<sup>★</sup>, Sebastian Borgeaud<sup>★</sup>, Arthur Mensch<sup>★</sup>, Elena Buchatskaya, Trevor Cai, Eliza Rutherford,  
Diego de Las Casas, Lisa Anne Hendricks, Johannes Welbl, Aidan Clark, Tom Hennigan, Eric Noland,  
Katie Millican, George van den Driessche, Bogdan Damoc, Aurelia Guy, Simon Osindero, Karen Simonyan,  
Erich Elsen, Jack W. Rae, Oriol Vinyals and Laurent Sifre<sup>★</sup>



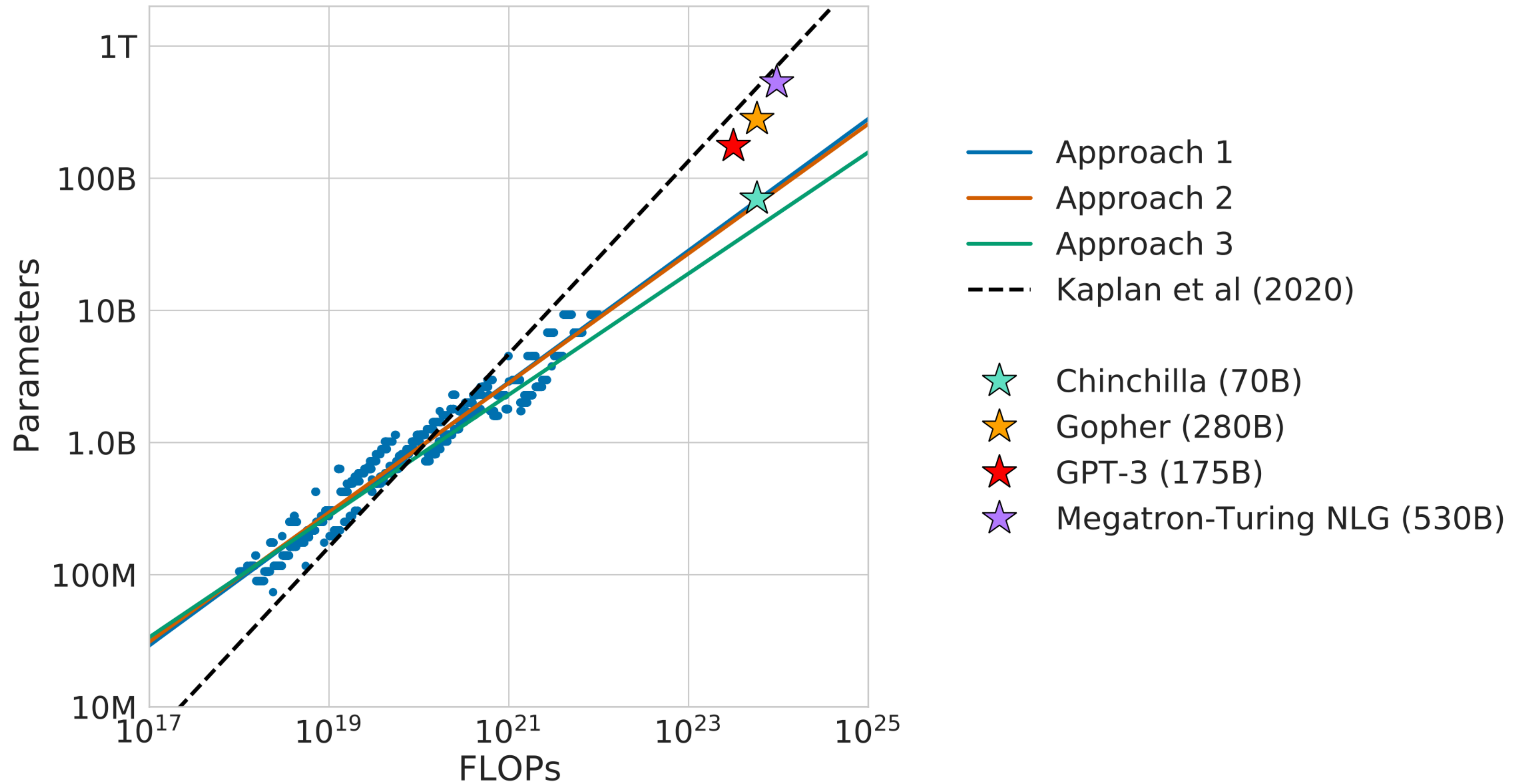
# Train longer on more tokens

## Lessons from training Chinchilla

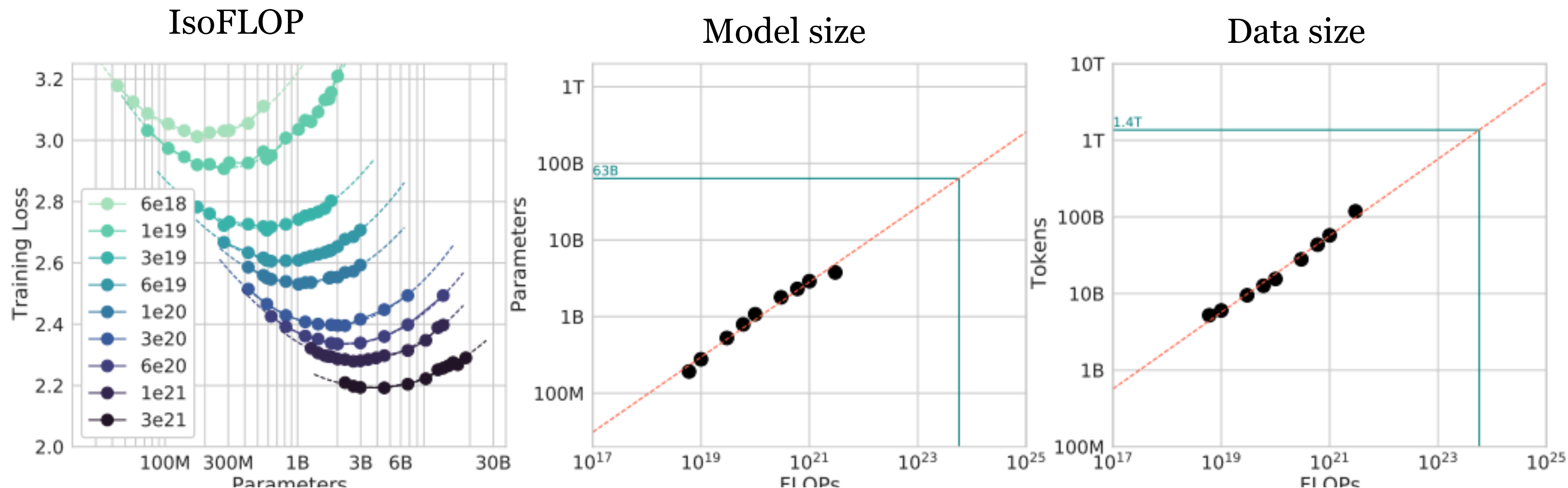
- From GPT3: large models should not be trained to lowest possible loss to be compute optimal
- Question: **Given a fixed FLOPs budget how should one trade off model size and number of training tokens?**
- Pre-training loss  $L(N, D)$  for  $N$  parameters and  $D$  training tokens. Find the optimal  $N$  and  $D$  values for a given compute budget.
- Empirical study on training 400 models from 70M to 16B parameters, trained on 5B to 400B tokens.
- Answer: **Train smaller models for (a lot) more training steps.**



- **Better to scale model size and number of tokens linearly!**



- For different model sizes, choose number of training tokens to keep FLOPs constant



**Figure 3 | IsoFLOP curves.** For various model sizes, we choose the number of training tokens such that the final FLOPs is a constant. The cosine cycle length is set to match the target FLOP count. We find a clear valley in loss, meaning that for a given FLOP budget there is an optimal model to train (**left**). Using the location of these valleys, we project optimal model size and number of tokens for larger models (**center** and **right**). In green, we show the estimated number of parameters and tokens for an *optimal* model trained with the compute budget of *Gopher*.

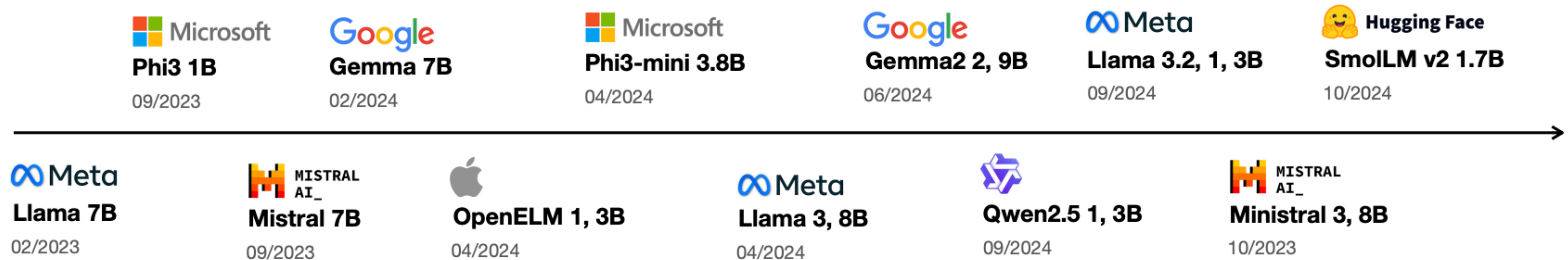
| Model                                              | Size (# Parameters) | Training Tokens |
|----------------------------------------------------|---------------------|-----------------|
| LaMDA ( <a href="#">Thoppilan et al., 2022</a> )   | 137 Billion         | 168 Billion     |
| GPT-3 ( <a href="#">Brown et al., 2020</a> )       | 175 Billion         | 300 Billion     |
| Jurassic ( <a href="#">Lieber et al., 2021</a> )   | 178 Billion         | 300 Billion     |
| <i>Gopher</i> ( <a href="#">Rae et al., 2021</a> ) | 280 Billion         | 300 Billion     |
| MT-NLG 530B ( <a href="#">Smith et al., 2022</a> ) | 530 Billion         | 270 Billion     |
| <i>Chinchilla</i>                                  | 70 Billion          | 1.4 Trillion    |



Is larger models always better?  
Can we go to even smaller models?



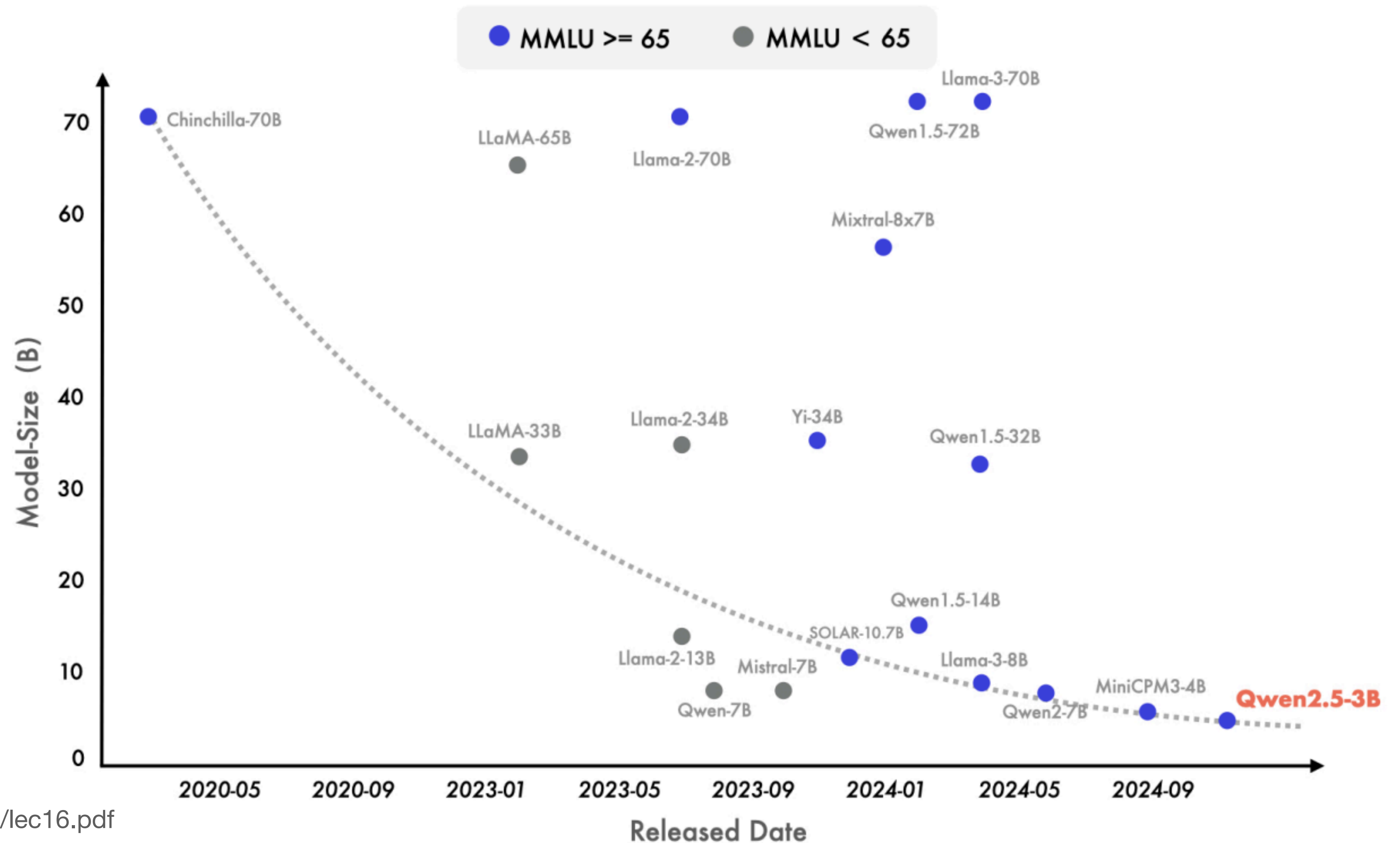
# Small language models



- SLM: language models <10B
- Fierce competition in small model regime
- Mostly open-weight

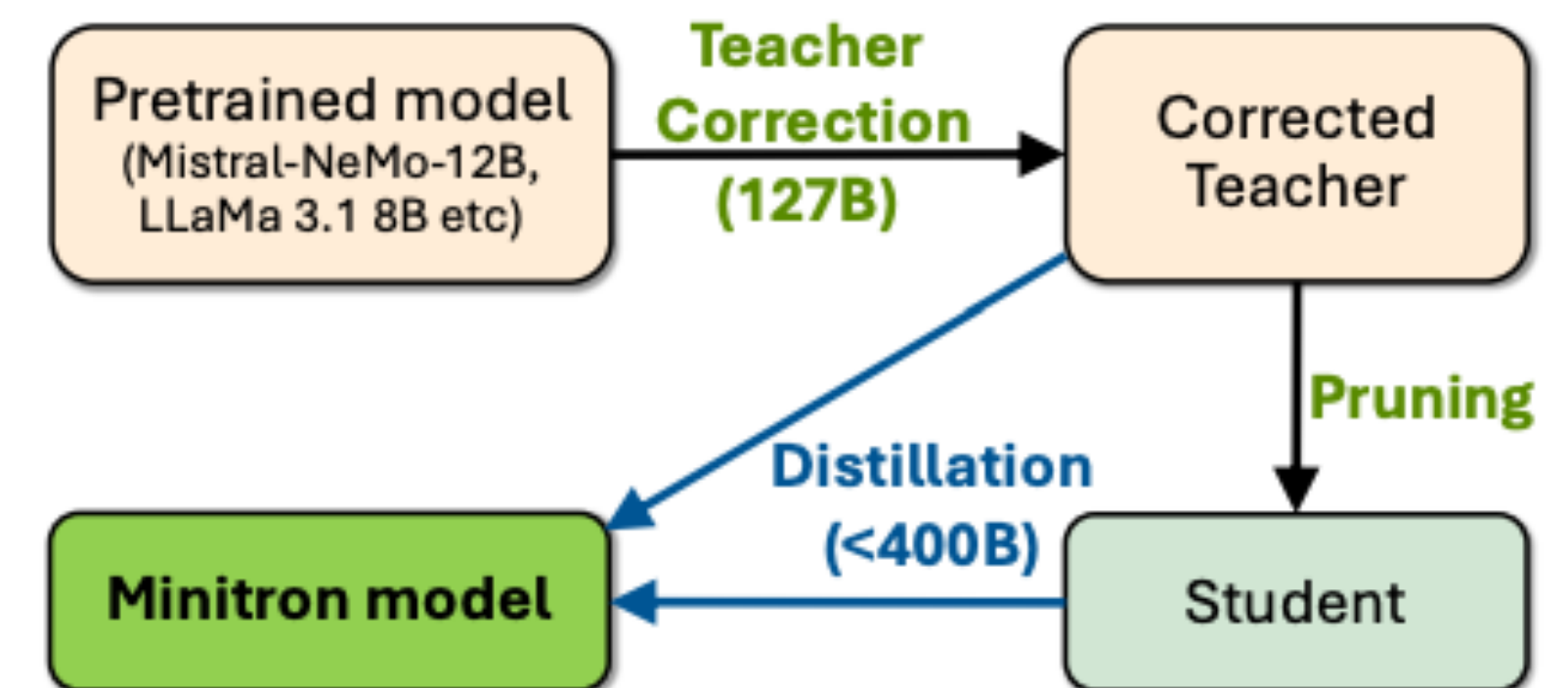
# Small language models are getting better and better

- Smaller models are also able to achieve good performance on MMLU



# How to obtain such small models?

- Train on **high quality data**
  - “If we have a small dataset is focused on **text-book quality educational content**, we can learn the task better, even with a smaller model.”
  - Phi-2: The surprising power of small language models
- Model **pruning** and **distillation**
  - **Pruning:** Sheared-Llama, Llama 3.2, Minitron
  - **Distillation:** Gemma2, Llama 3.2, Minitron



Minitron



# Types of pruning

- Unstructured pruning
  - Magnitude pruning
  - Weights and activations
- Structured pruning
  - Prune entire blocks / components

|                                | $U$          | $T$          | $\nearrow$    | Params | MNLI |
|--------------------------------|--------------|--------------|---------------|--------|------|
| BERT <sub>base</sub> (teacher) | $\times$     | $\times$     | 1.0 $\times$  | 85M    | 84.8 |
| <b>Distillation</b>            |              |              |               |        |      |
| DistillBERT <sub>6</sub>       | $\checkmark$ | $\times$     | 2.0 $\times$  | 43M    | 82.2 |
| TinyBERT <sub>6</sub>          | $\checkmark$ | $\checkmark$ | 2.0 $\times$  | 43M    | 84.0 |
| MobileBERT <sup>†</sup>        | $\checkmark$ | $\times$     | 2.3 $\times$  | 20M    | 83.9 |
| DynaBERT                       | $\times$     | $\checkmark$ | 6.3 $\times$  | 11M    | 76.3 |
| AutoTinyBERT <sup>†</sup>      | $\checkmark$ | $\checkmark$ | 9.1 $\times$  | 3.3M   | 78.2 |
| TinyBERT <sub>4</sub>          | $\checkmark$ | $\checkmark$ | 11.4 $\times$ | 4.7M   | 78.8 |
| <b>Pruning</b>                 |              |              |               |        |      |
| Movement Pruning               | $\times$     | $\checkmark$ | 1.0 $\times$  | 9M     | 81.2 |
| Block Pruning                  | $\times$     | $\checkmark$ | 2.7 $\times$  | 25M    | 83.7 |
| CoFi Pruning (ours)            | $\times$     | $\checkmark$ | 2.7 $\times$  | 26M    | 84.9 |
| CoFi Pruning (ours)            | $\times$     | $\checkmark$ | 12.1 $\times$ | 4.4M   | 80.6 |

Table 1: A comparison of state-of-the-art distillation and pruning methods.  $U$  and  $T$  denote whether *Unlabeled* and *Task-specific* are used for distillation or pruning. The inference speedups ( $\nearrow$ ) are reported against a BERT<sub>base</sub> model and we evaluate all the models on an NVIDIA V100 GPU (§4.1). The models labeled as <sup>†</sup> use a different teacher model and are not a direct comparison. Models are one order of magnitude faster.<sup>3</sup>

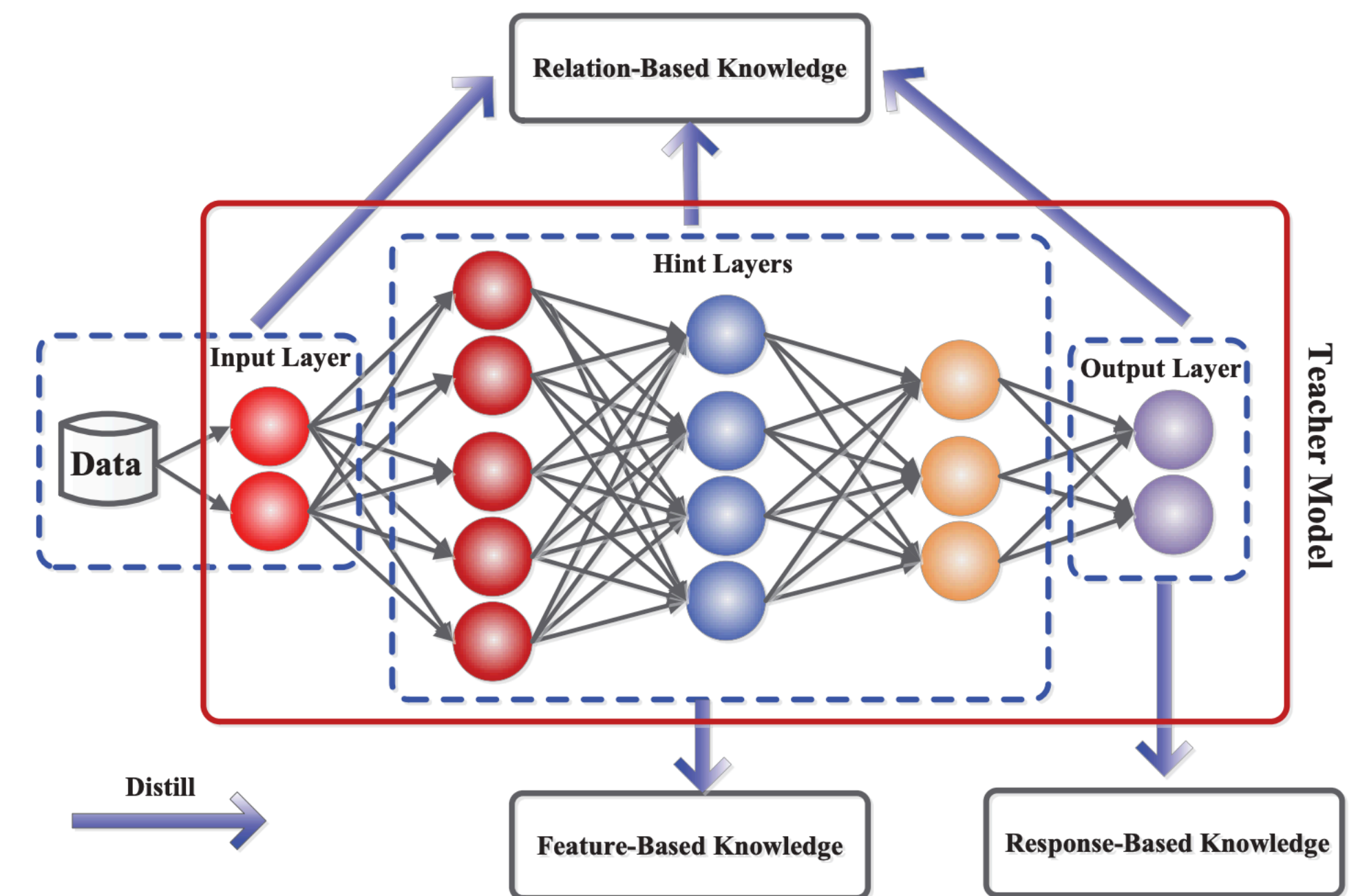


# Different ways to distill

- Response-based
  - Try to match output (either predictions or logits)
- Feature-based  $L_{FeaD}(f_t(x), f_s(x)) = \mathcal{L}_F(\Phi_t(f_t(x)), \Phi_s(f_s(x)))$ 
  - Try to match feature activation
  - Can be at different hidden layers
- Relation-based
  - Relationship between different features

$$L_{RelD}(f_t, f_s) = \mathcal{L}_{R^1}(\Psi_t(\hat{f}_t, \check{f}_t), \Psi_s(\hat{f}_s, \check{f}_s))$$

- Phi: Transformation functions to map features to the same shape



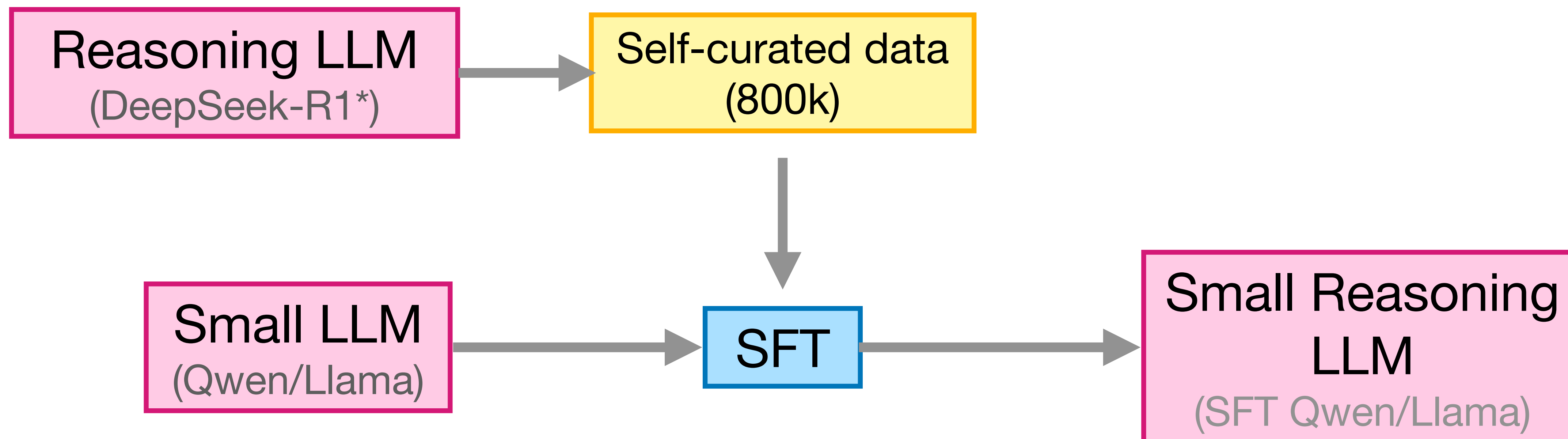
Gou et al. 2020

- Distillation losses can be L\_2, CE, MMD, KL

# Distillation of DeepSeek-R1

## Training recipe

Generated / Filtered using DeepSeek-R1\*  
Reasoning (600K)  
General (200K)



# Distillation

| Model                         | AIME 2024   |             | MATH-500    | GPQA<br>Diamond | LiveCode<br>Bench | CodeForces  |
|-------------------------------|-------------|-------------|-------------|-----------------|-------------------|-------------|
|                               | pass@1      | cons@64     | pass@1      | pass@1          | pass@1            | rating      |
| GPT-4o-0513                   | 9.3         | 13.4        | 74.6        | 49.9            | 32.9              | 759         |
| Claude-3.5-Sonnet-1022        | 16.0        | 26.7        | 78.3        | 65.0            | 38.9              | 717         |
| OpenAI-o1-mini                | 63.6        | 80.0        | 90.0        | 60.0            | 53.8              | <b>1820</b> |
| QwQ-32B-Preview               | 50.0        | 60.0        | 90.6        | 54.5            | 41.9              | 1316        |
| DeepSeek-R1-Distill-Qwen-1.5B | 28.9        | 52.7        | 83.9        | 33.8            | 16.9              | 954         |
| DeepSeek-R1-Distill-Qwen-7B   | 55.5        | 83.3        | 92.8        | 49.1            | 37.6              | 1189        |
| DeepSeek-R1-Distill-Qwen-14B  | 69.7        | 80.0        | 93.9        | 59.1            | 53.1              | 1481        |
| DeepSeek-R1-Distill-Qwen-32B  | <b>72.6</b> | 83.3        | 94.3        | 62.1            | 57.2              | 1691        |
| DeepSeek-R1-Distill-Llama-8B  | 50.4        | 80.0        | 89.1        | 49.0            | 39.6              | 1205        |
| DeepSeek-R1-Distill-Llama-70B | 70.0        | <b>86.7</b> | <b>94.5</b> | <b>65.2</b>     | <b>57.5</b>       | 1633        |

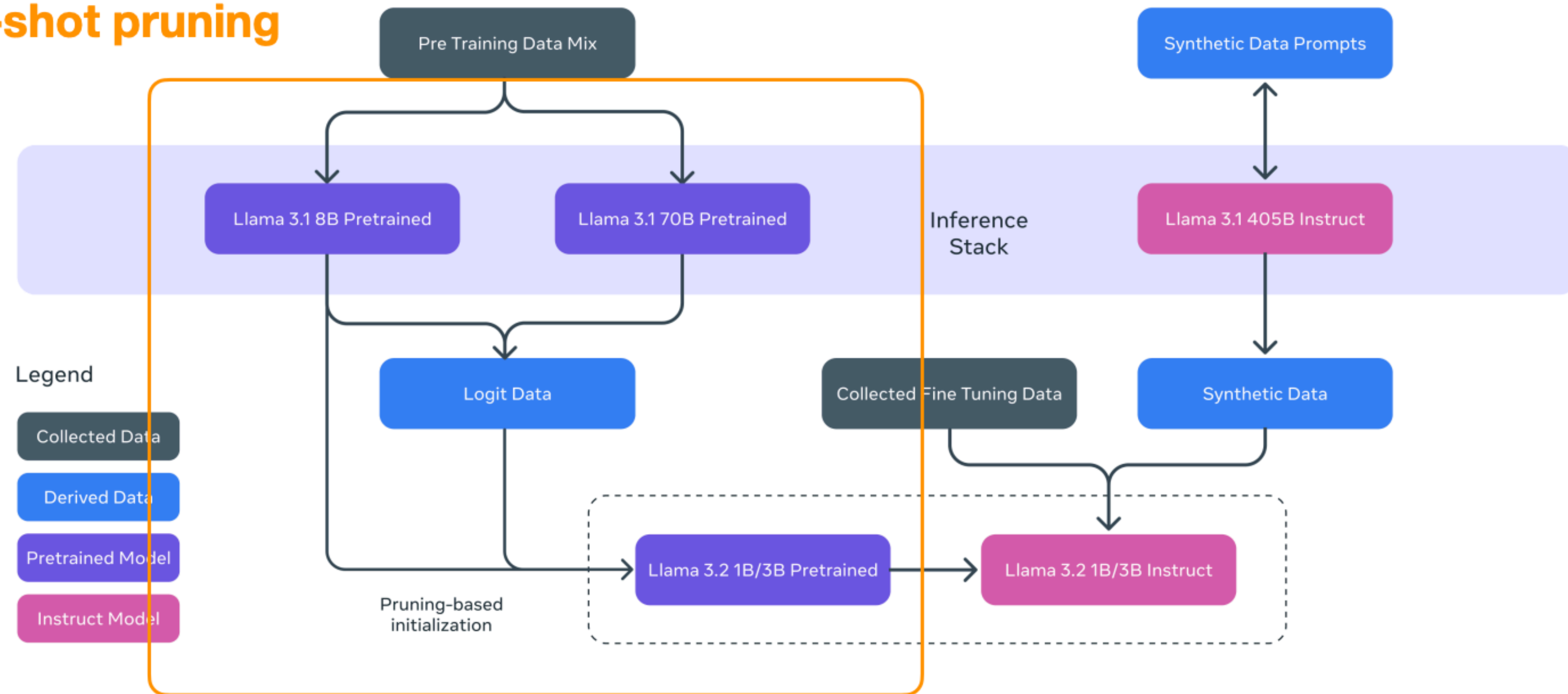
Table 5 | Comparison of DeepSeek-R1 distilled models and other comparable models on reasoning-related benchmarks.



# Combined pruning and distillation

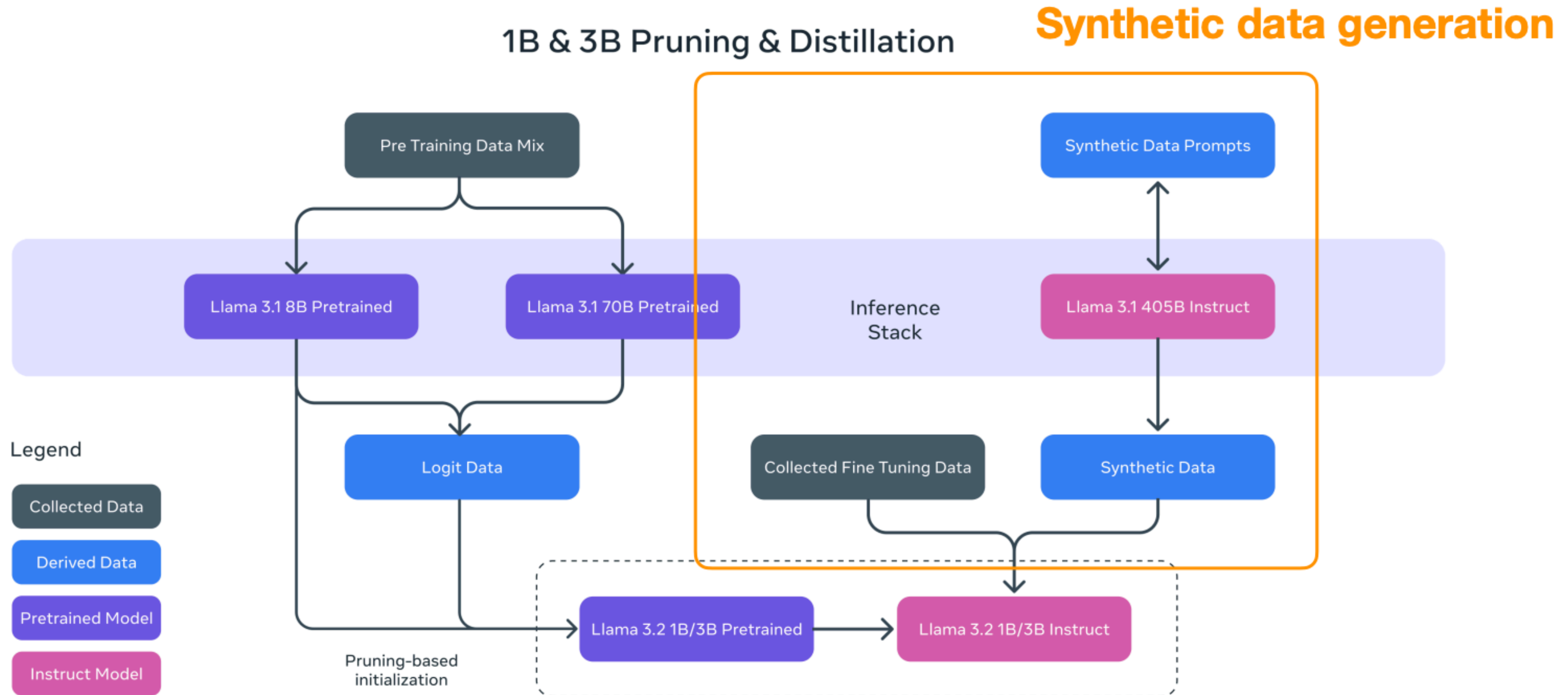
## 1B & 3B Pruning & Distillation

### One-shot pruning





# Combined pruning and distillation



# Test-time scaling

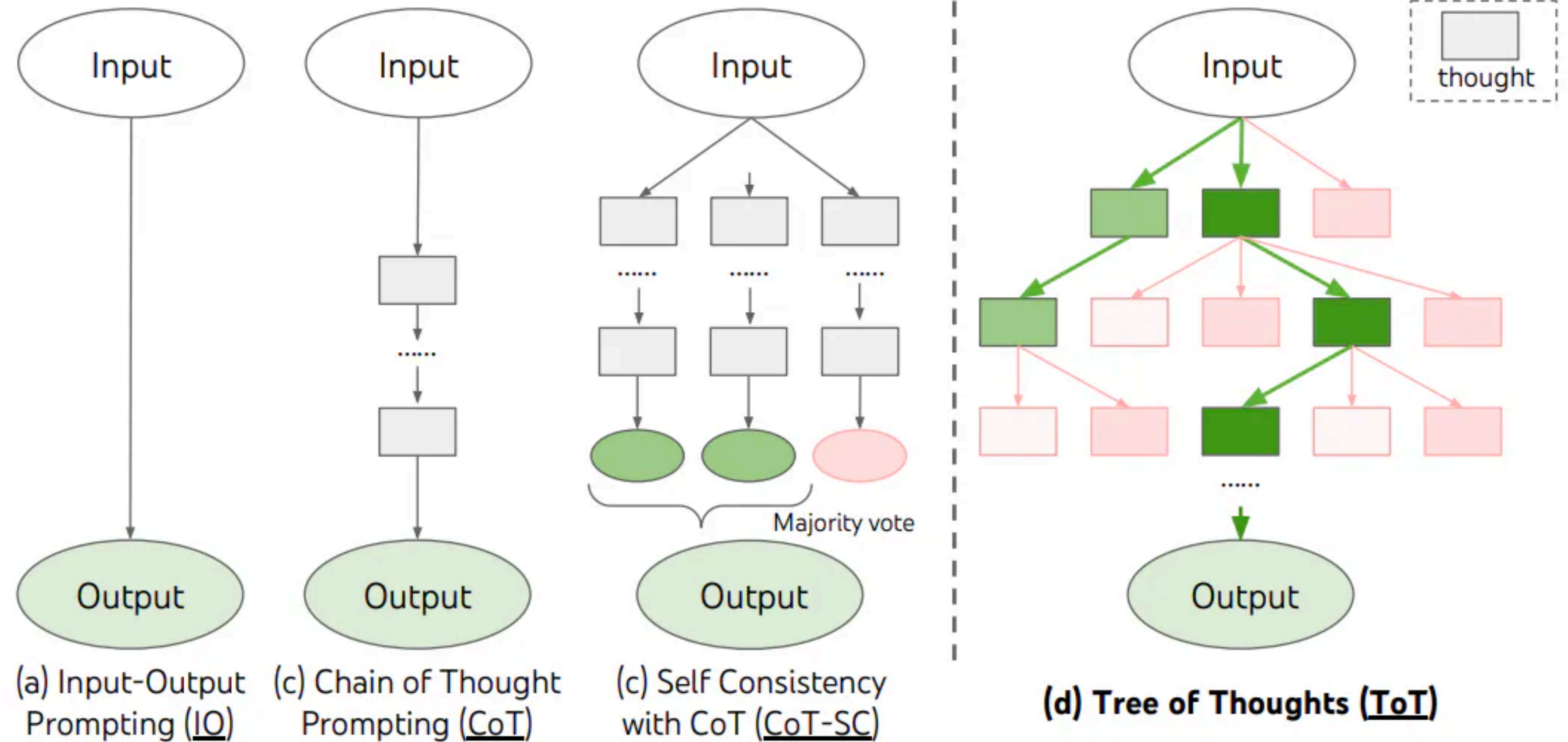
# Test time scaling

- Scaling laws for training
- What about scaling laws for inference?
  - Tradeoff model size vs inference time compute
    - Large models can only be deployed on machines with large memory
    - Smaller models can be deployed on smaller devices
- Generate multiple output chains and do search to select / aggregate to form a final output



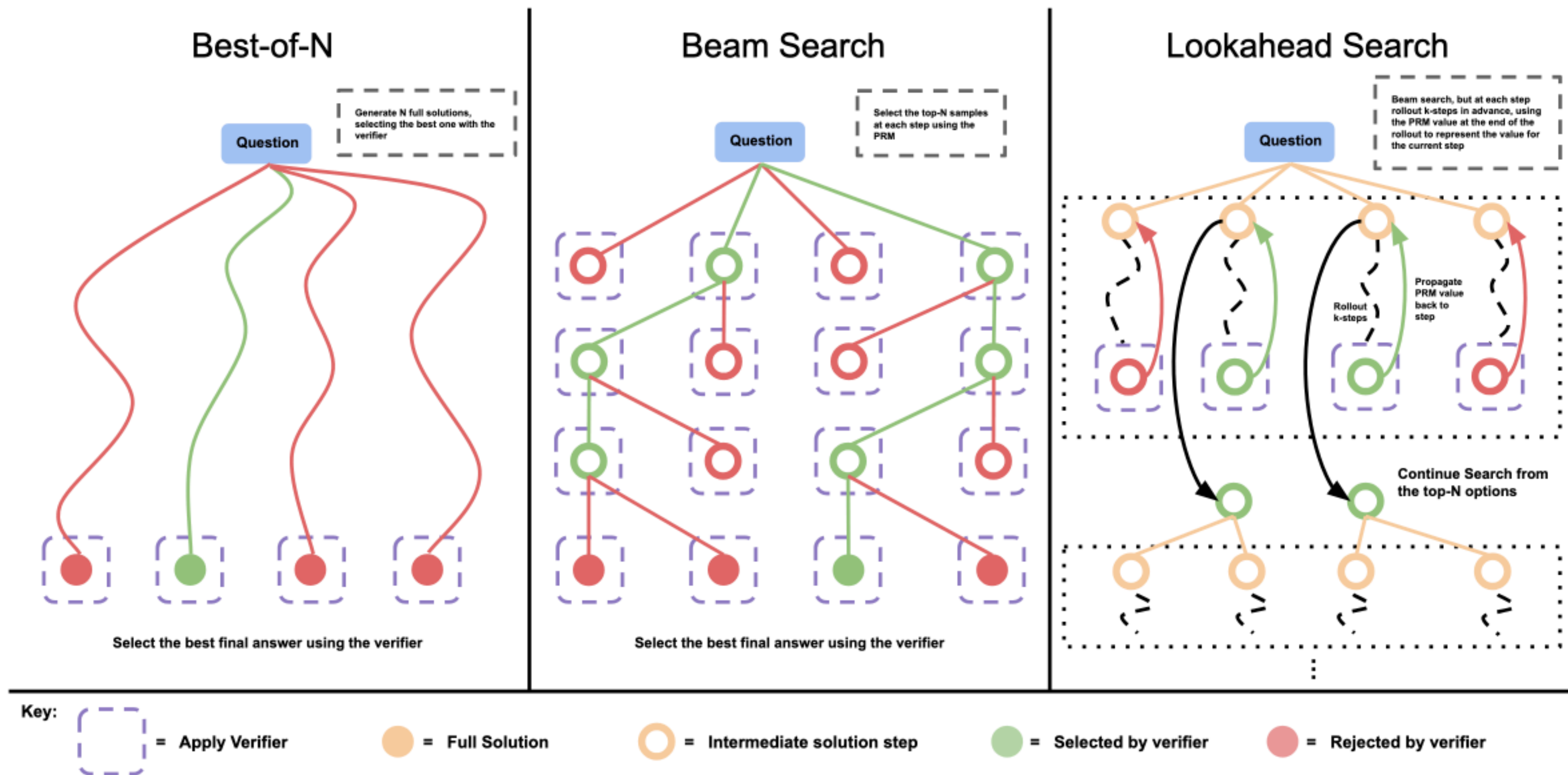
# Tree of thought

- Decision making by considering multiple paths of reasoning
- Consider different “thoughts” expressed in language
- Use to solve different types of problems



*Tree of Thoughts: Deliberate Problem Solving with Large Language Models [Yao et al, 2023]*  
*Large Language Model Guided Tree-of-Thought [Long 2023]*





**Figure 2 | Comparing different PRM search methods.** **Left:** Best-of-N samples N full answers and then selects the best answer according to the PRM final score. **Center:** Beam search samples N candidates at each step, and selects the top M according to the PRM to continue the search from. **Right:** lookahead-search extends each step in beam-search to utilize a k-step lookahead while assessing which steps to retain and continue the search from. Thus lookahead-search needs more compute.

